Introduction aux réseaux en biologie

Bioinformatique I (BIF-7900)

Antoine Bodein

19 novembre 2026



1

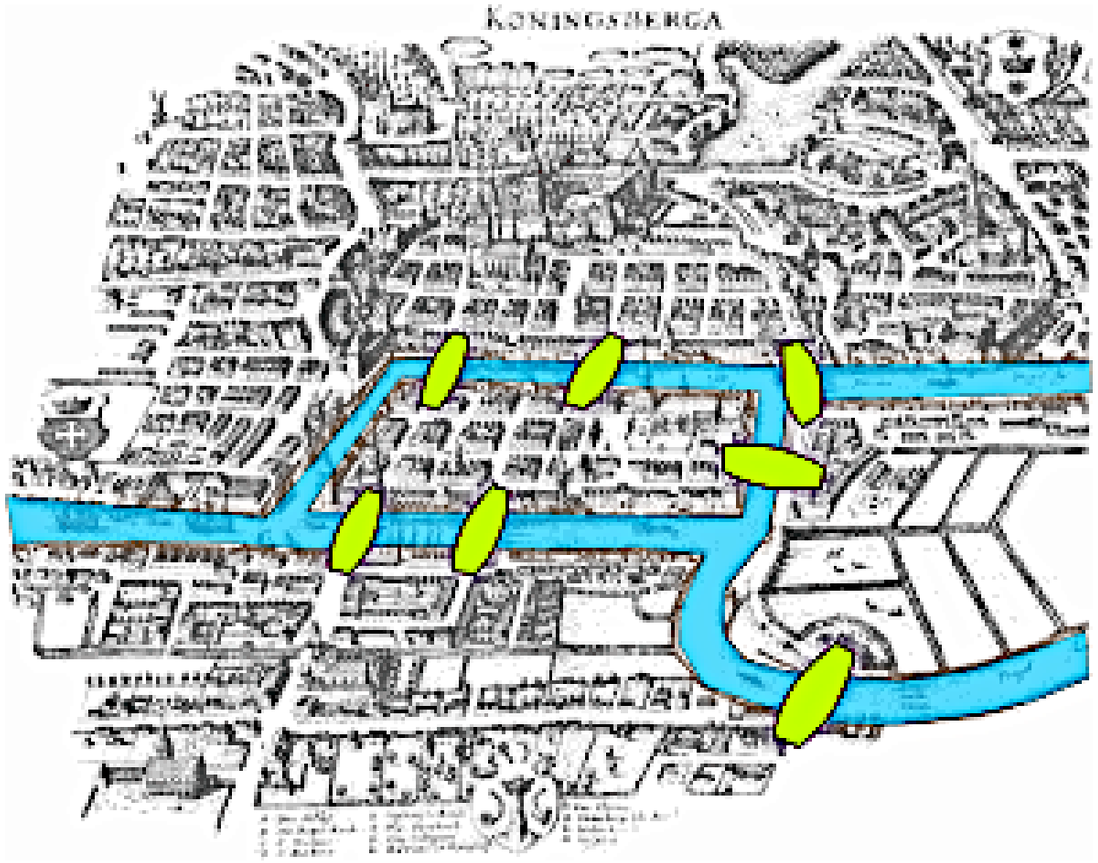
INTRODUCTION

Objectifs

1. Comprendre ce qu'est un **réseau** (un graphe) et choisir la bonne représentation
2. Connaître les principaux **réseaux biologiques** et d'où viennent leurs liens
3. Calculer et interpréter les **descripteurs topologiques** : degré, chemins, centralité, *clustering*
4. Trouver des **modules** (communautés) et savoir ce qu'ils disent… et ne disent pas
5. Manipuler des réseaux dans R avec **igraph**

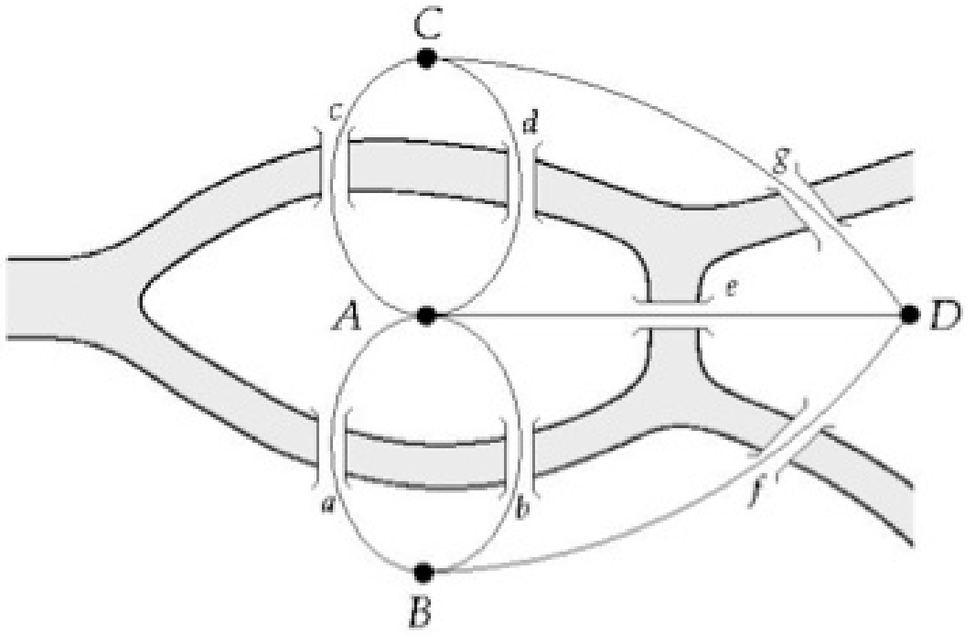
> Fil conducteur, comme aux modules précédents : les **erreurs silencieuses**. Un graphe dirigé sans le savoir, des poids lus comme des distances, un seuil arbitraire, un dessin pris pour un résultat.

Les sept ponts de Königsberg



Königsberg (aujourd'hui Kaliningrad), XVIIIe siècle : la Pregel, deux îles, **sept ponts**.

Peut-on se promener en traversant chaque pont une et une seule fois ?



Quatre terres (A, B, C, D), sept ponts : le premier **graphe**. Barabási, *Network Science*, chap. 2.

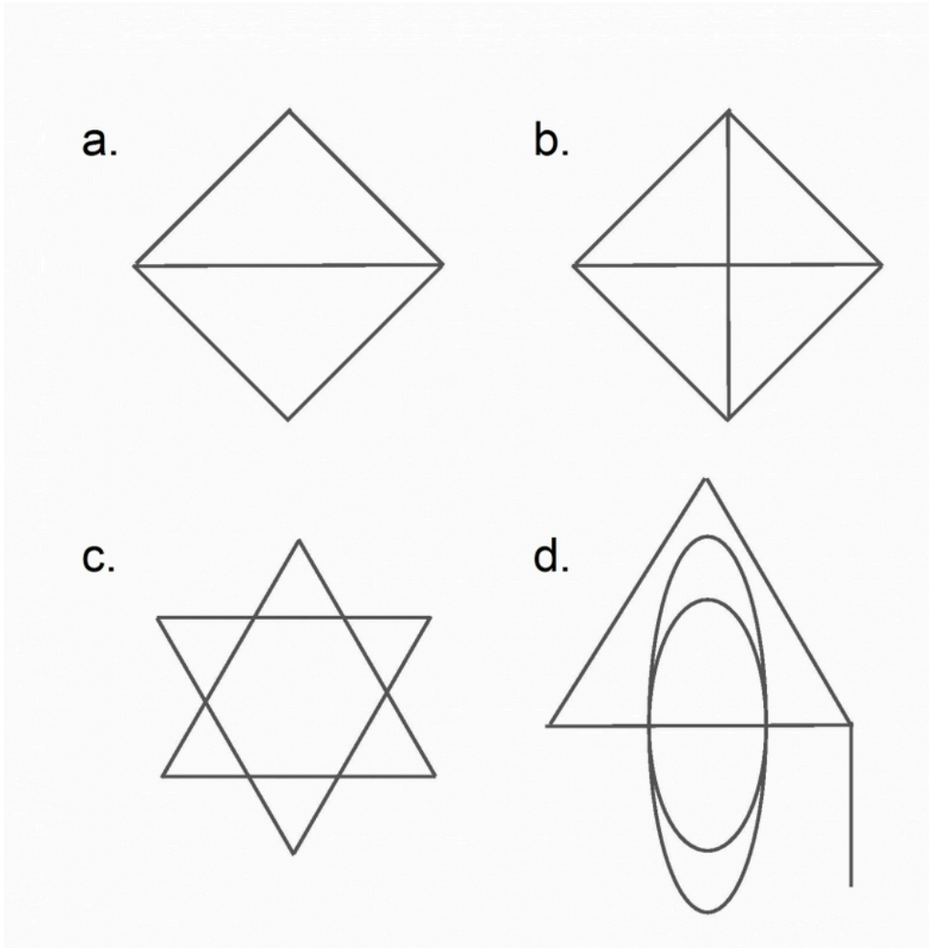
Le théorème d'Euler (1736)

- Ce qui compte n'est ni la longueur des ponts ni la forme des îles : seulement **qui est relié à qui**
- On entre dans une terre et on en ressort : un nœud traversé utilise ses liens **par paires**
- Seuls le **départ** et l'**arrivée** peuvent avoir un nombre **impair** de liens

- **Plus de deux** nœuds de degré impair : le chemin est **impossible**
- Graphe connexe, **zéro ou deux** nœuds de degré impair : le chemin **existe**

> À Königsberg, les **quatre** terres ont un nombre impair de ponts (5, 3, 3, 3). Euler l'a démontré sans marcher : en changeant de **représentation**.

✋ À vous : sans lever le crayon



Quelles figures peut-on dessiner **sans lever le crayon** et sans repasser deux fois sur un trait ? Pourquoi ?

- **a.** Oui : deux nœuds de degré **3** (les extrémités de la diagonale), deux de degré 2 ; on part d'une extrémité de la diagonale, on finit à l'autre
- **b.** Non : quatre coins de degré **3**, impairs
- **c.** Oui : tous les degrés sont pairs
- **d.** Oui : au plus **deux** nœuds impairs

On ne trace rien : on **compte les degrés**.

Plan

1. Introduction
2. Qu’est-ce qu’un réseau ?
3. Représenter un réseau dans R → **TP1**
4. Les réseaux en biologie
5. Propriétés topologiques
6. Modules et communautés → **TP2**

2

QU’EST-CE QU’UN RÉSEAU ?

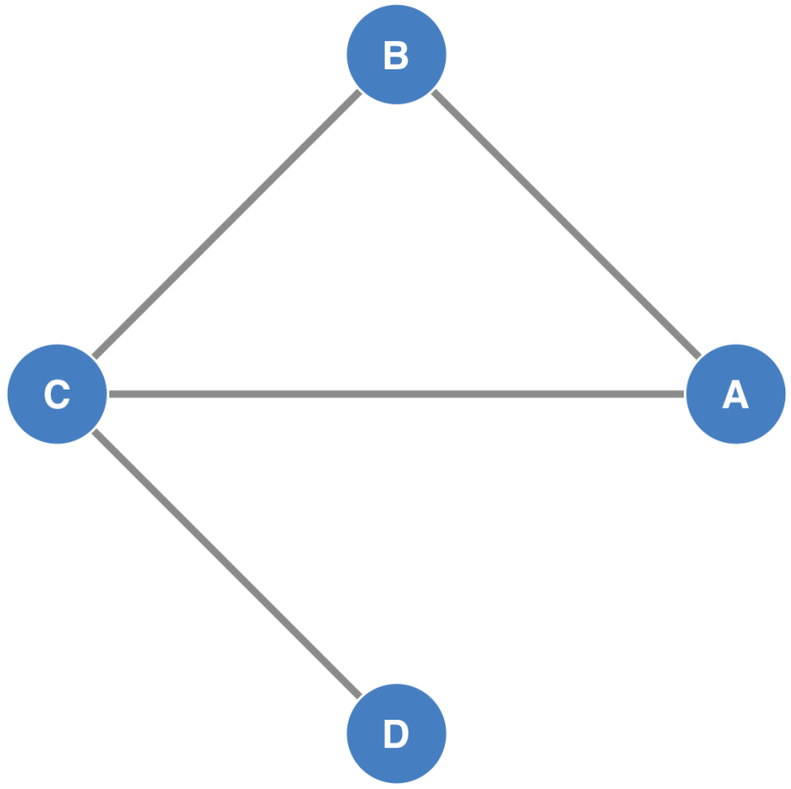
Les éléments constitutifs

	Français	Anglais	Notation
Les composants	**nœuds**, sommets	*nodes*, *vertices*	N
Les interactions	**liens**, arêtes	*links*, *edges*	L
Le système	**réseau**, graphe	*network*, *graph*	(N, L)

« Réseau » désigne plutôt le système réel (des protéines qui interagissent), « graphe » sa représentation mathématique. igraph parle de *vertices* (`V()`) et d'*edges* (`E()`).

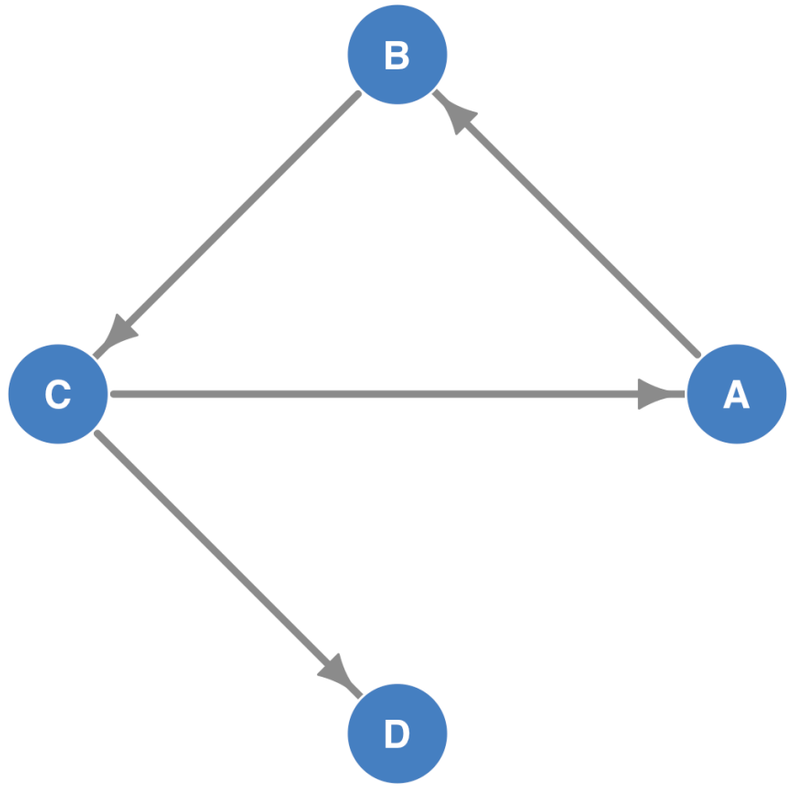
Barabási, *Network Science* (2016), chap. 2 : http://networksciencebook.com.

Dirigé ou non dirigé



Non dirigé : le lien est **symétrique**

Interaction physique protéine-protéine, co-expression, co-apparition dans une scène



Dirigé (*digraph*) : le lien a un **sens** (un arc)

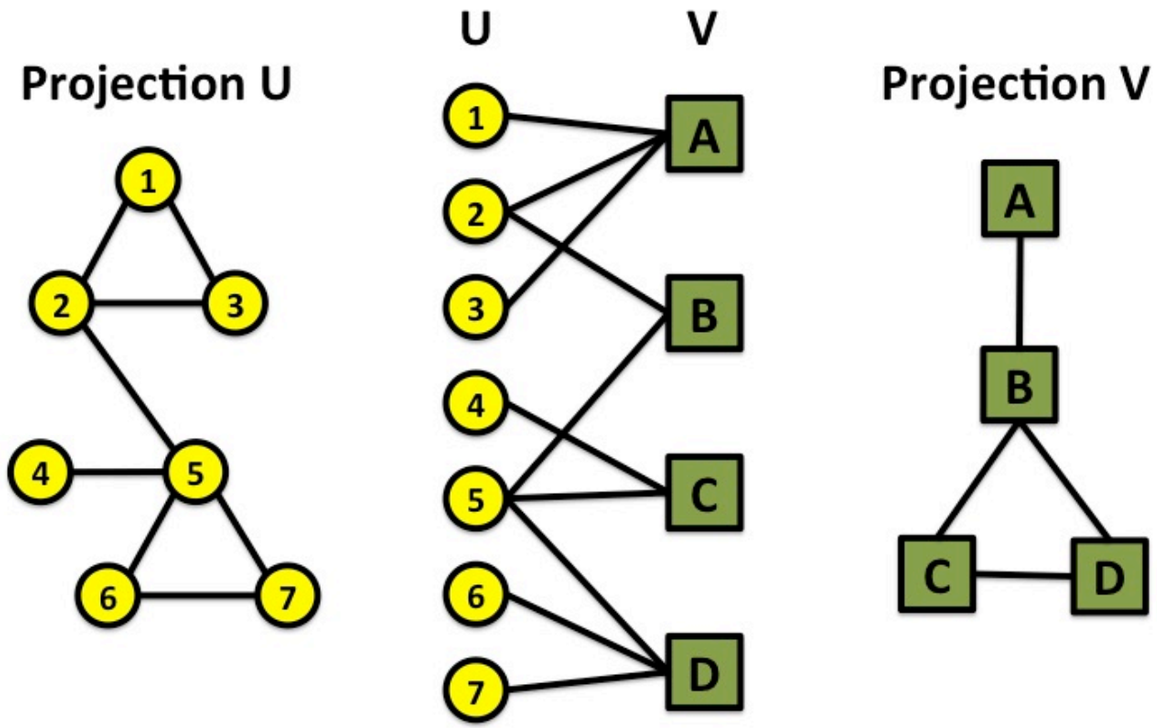
Un facteur de transcription régule un gène, une réaction produit un métabolite, une page web cite une autre

Pondéré, boucles, multigraphe

- **Pondéré** : chaque lien porte une valeur (nombre d'interactions, corrélation, score de confiance)
- **Boucle** (*self-loop*) : un nœud relié à lui-même (un gène qui régule sa propre expression)
- **Multigraphe** : plusieurs liens entre les mêmes nœuds (deux types d'interaction, ou la même interaction détectée deux fois)

> Le **sens** d'un poids n'est pas écrit dans le graphe. 192 scènes partagées entre Ned et Robert : est-ce une **force** (proches) ou un **coût** (loin) ? R, lui, a son idée (section 5).

Les réseaux bipartites

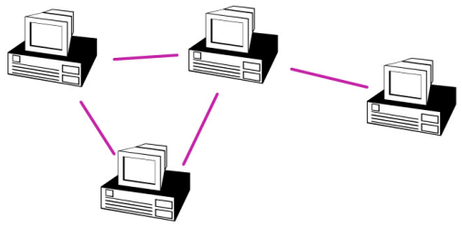


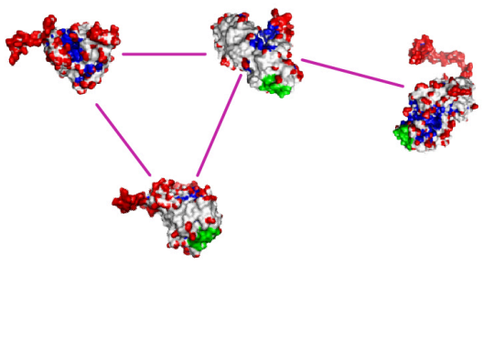
Deux **types** de nœuds (U et V) ; chaque lien relie un nœud de U à un nœud de V, jamais deux nœuds du même type.

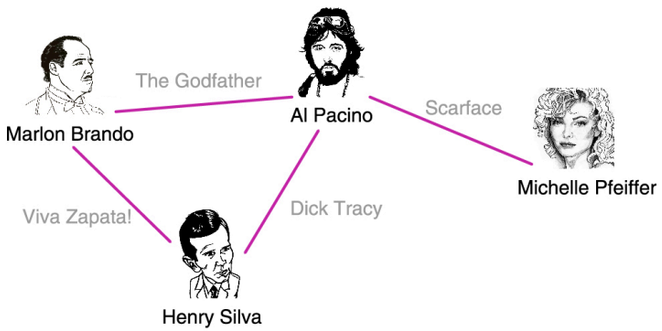
- gènes ↔ maladies
- métabolites ↔ réactions
- gènes ↔ termes GO

La **projection** sur V relie deux nœuds de V qui partagent un voisin dans U : on y **perd** de l'information.

Un même graphe, plusieurs réseaux







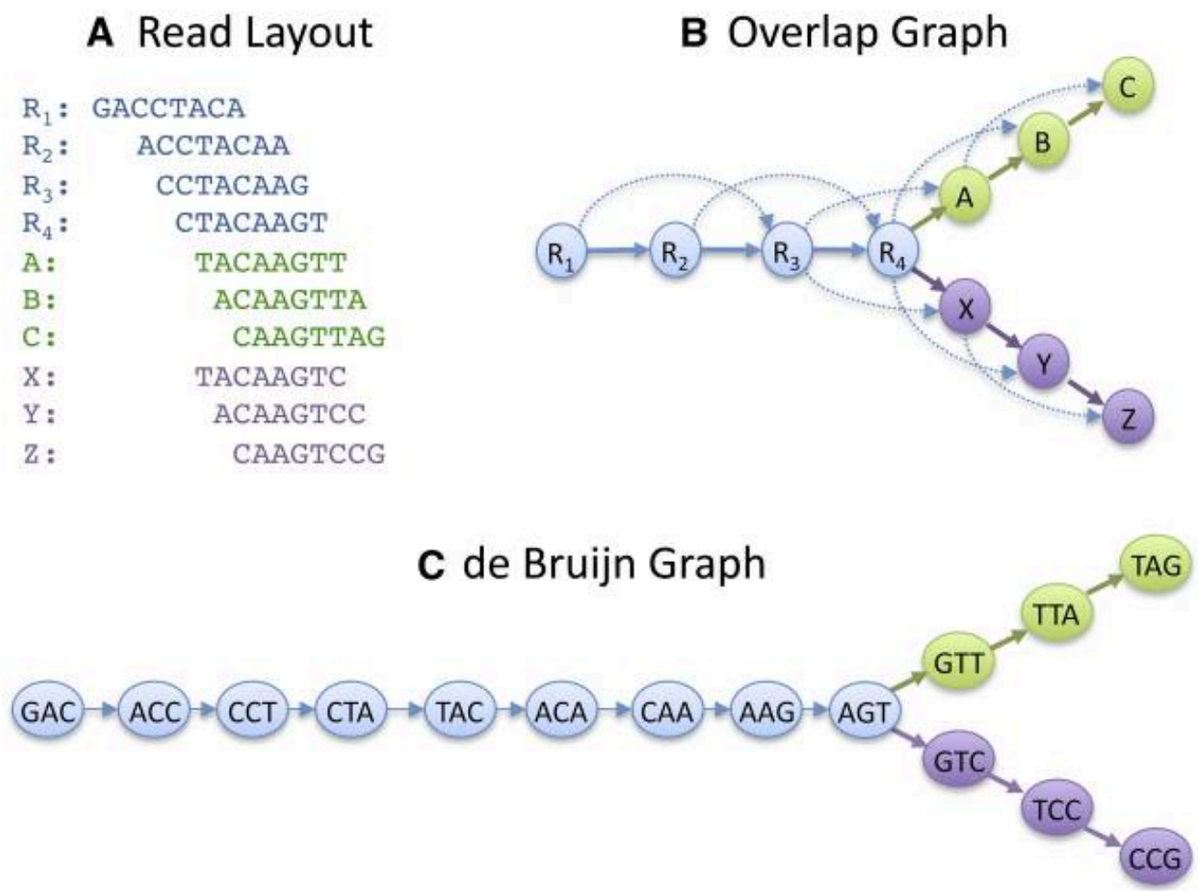
N = 4, L = 4 : **le même graphe** pour

- des acteurs qui ont tourné ensemble
- des ordinateurs reliés par des câbles
- des protéines qui se lient

Les mêmes outils (degré, chemins, modules) s'appliquent partout.
L'interprétation, elle, dépend du système.

Barabási, *Network Science*, chap. 2.

Choisir la bonne représentation



Assembler un génome à partir de lectures :

- **graphe de chevauchement** : un nœud par lecture, un lien si deux lectures se chevauchent ; reconstruire = un chemin **hamiltonien** (difficile)
- **graphe de de Bruijn** : un nœud par *k*-mer, un lien par chevauchement de $k-1$; reconstruire = un chemin **eulérien** (facile)

> Mêmes données, deux graphes, deux problèmes de difficulté très différente : c'est le pont de Königsberg qui fait tourner les assembleurs actuels (Compeau *et al.*, 2011).

✋ À vous : trouver un réseau

Par deux, en 3 minutes : un **réseau réel** (biologique ou non).

1. Que représentent les **nœuds** ?
2. Que représentent les **liens** ?
3. Le graphe est-il **dirigé** ? **pondéré** ? Que signifie le poids ?
4. Y a-t-il des **boucles**, des liens **multiples**, deux **types** de nœuds ?

Exemples : réseau métabolique (dirigé, bipartite métabolites-réactions), co-auteurs (non dirigé, pondéré par le nombre d'articles), régulation génique (dirigé, boucles d'autorégulation), transport aérien (dirigé, pondéré par le nombre de vols), réseau trophique (dirigé : qui mange qui).

3

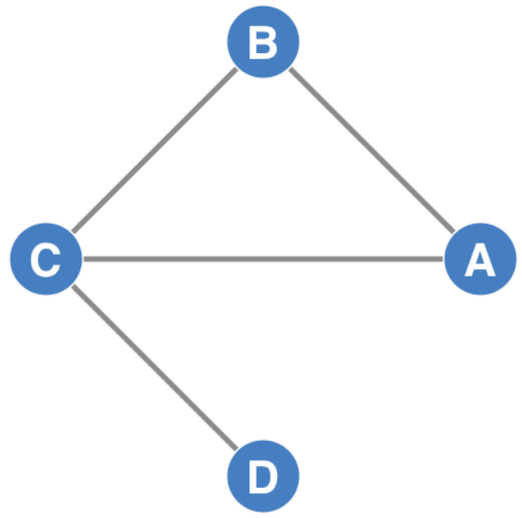
REPRÉSENTER UN RÉSEAU DANS R

Deux façons d'écrire un graphe

Une **liste d'arêtes** : une ligne par lien. C'est la forme la plus courante dans un fichier.

liste d'arêtes

from	to
A	B
A	C
B	C
C	D



Deux façons d'écrire un graphe

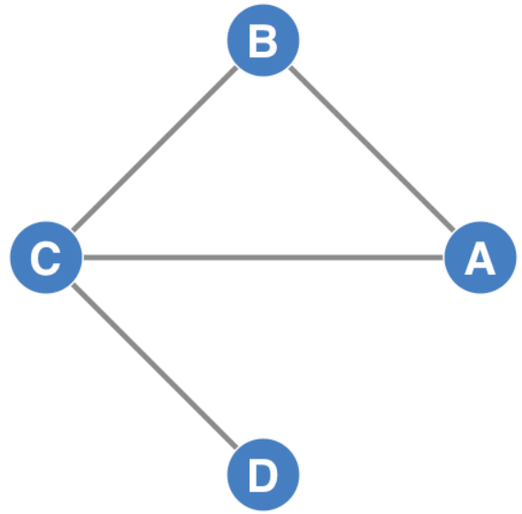
Matrice d'adjacence : $A_{ij} = 1$ s'il existe un lien entre i et j. Le lien A–B remplit **deux** cases : la matrice est **symétrique**.

liste d'arêtes

from	to
A	B
A	C
B	C
C	D

matrice d'adjacence

	A	B	C	D
A	0	1	0	0
B	1	0	0	0
C	0	0	0	0
D	0	0	0	0



Deux façons d'écrire un graphe

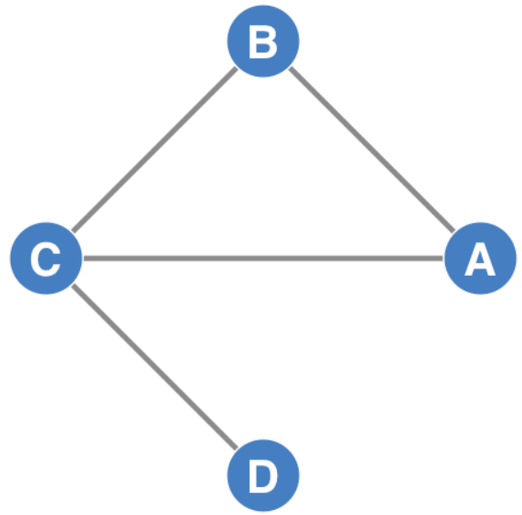
Les quatre liens : 8 cases à 1 sur 16. Diagonale nulle (pas de boucle). La **somme d'une ligne** est le **degré** du nœud.

liste d'arêtes

from	to
A	B
A	C
B	C
C	D

matrice d'adjacence

	A	B	C	D
A	0	1	1	0
B	1	0	1	0
C	1	1	0	1
D	0	0	1	0

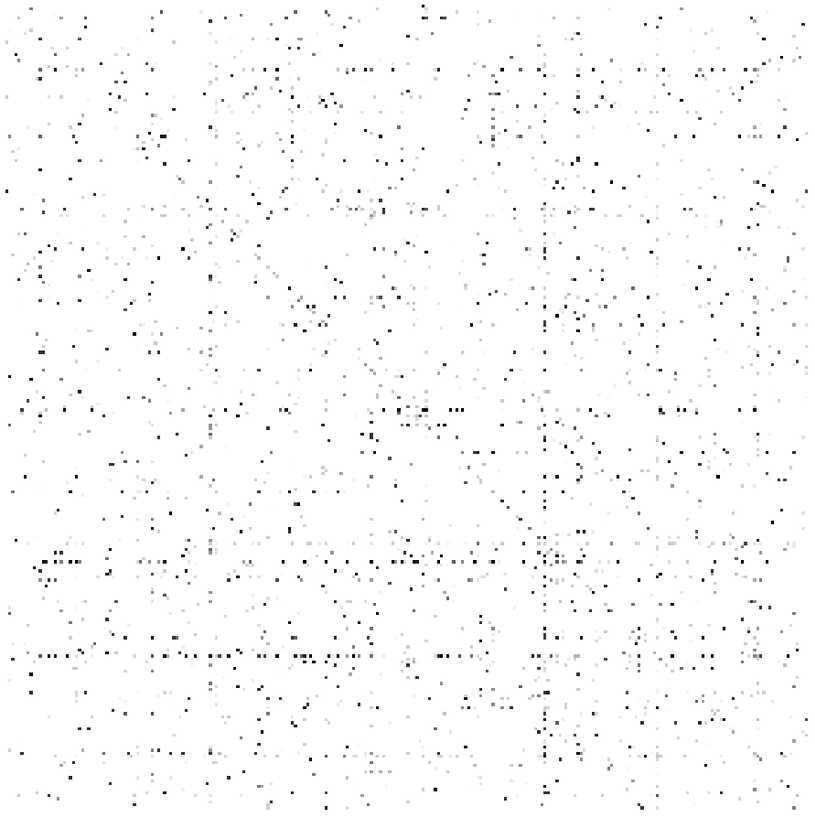


Dirigé : une matrice non symétrique

```
as_adjacency_matrix(g_jouet, sparse = FALSE)
#>   A B C D
#> A 0 1 1 0
#> B 1 0 1 0
#> C 1 1 0 1
#> D 0 0 1 0
g_dir <- graph_from_data_frame(jouet, directed = TRUE)
as_adjacency_matrix(g_dir, sparse = FALSE)
#>   A B C D
#> A 0 1 1 0
#> B 0 0 1 0
#> C 0 0 0 1
#> D 0 0 0 0
```

- Dirigé : A_{ij} = 1 s'il existe un lien **de** i **vers** j. La ligne donne les liens **sortants**, la colonne les **entrants**
- Pondéré : A_{ij} = le **poids** au lieu de 1

Les réseaux réels sont creux



Matrice d'adjacence du réseau d'interactions protéiques de la levure : un point par lien (Barabási, *Network Science*, chap. 2).

- N nœuds : N(N−1)/2 liens **possibles** (non dirigé)
- **Densité** = L / [N(N−1)/2]
- 20 000 gènes : 200 millions de paires possibles, quelques centaines de milliers d'interactions connues

```
edge_density(got)
#> [1] 0.06874141
```

Game of Thrones : 550 liens sur 8001 possibles. La matrice est remplie de **0** : on stocke plutôt la **liste** d'arêtes (ou une matrice *creuse*).

Les données du jour : Game of Thrones

```
got_aretes <- read_csv("/home/public/reseaux/got_edges.csv")
got_sexe   <- read_csv("/home/public/reseaux/got_sex.csv")
```

```
got_aretes
#> # A tibble: 550 × 3
#>   from     to           weight
#>   <chr>    <chr>         <dbl>
#> 1 Ned      Robert          192
#> 2 Daenerys Jorah           154
#> 3 Jon      Sam             121
#> 4 Ned      Littlefinger    107
#> 5 Ned      Varys            96
#> 6 Daenerys Drogo            91
#> # ℹ 544 more rows
```

Saison 1 : deux personnages sont liés quand l'un parle juste après l'autre, parle de l'autre, quand un tiers parle des deux, quand ils sont nommés dans la même didascalie ou apparaissent dans la même scène. `weight` = nombre de fois. Données d'A. Beveridge (*Network of Thrones*), via le package `networkdata` (D. Schoch).

graph_from_data_frame()

```
graph_from_data_frame(got_aretes, directed = FALSE, vertices = got_sexe)
```

les 2 premières colonnes : les extrémités ; les autres : attributs des liens

TRUE par défaut !

facultatif : un nœud par ligne, 1re colonne = le nom

```
got <- graph_from_data_frame(got_aretes, directed = FALSE, vertices = got_sexe)
got
#> IGRAPH 3874741 UNW- 127 550 --
#> + attr: name (v/c), sex (v/c), weight (e/n)
#> + edges from 3874741 (vertex names):
#>  [1] Ned         --Robert        Daenerys    --Jorah
#>  [3] Jon         --Sam           Ned         --Littlefinger
#>  [5] Ned         --Varys         Daenerys    --Drogo
#>  [7] Ned         --Arya          Catelyn     --Robb
#>  [9] Bronn       --Tyrion        Ned         --Cersei
#> [11] Cersei      --Robert        Littlefinger--Varys
#> [13] Shae        --Tyrion        Ned         --Catelyn
#> [15] Joffrey     --Sansa         Ned         --Pycelle
#> + ... omitted several edges
```

Lire l'en-tête d'un objet igraph

```
#> IGRAPH 3874741 UNW- 127 550 --
#> + attr: name (v/c), sex (v/c), weight (e/n)
```

Code	Sens
`U` / `D`	**U**ndirected / **D**irected
`N`	les nœuds ont un attribut `name` (**N**amed)
`W`	les liens ont un attribut `weight` (**W**eighted)
`B`	**B**ipartite (ici `–` : non)
`127 550`	N = 127 nœuds, L = 550 liens
`name (v/c)`	attribut de **v**ertex, de type **c**aractère ; `(e/n)` : d'**e**dge, **n**umérique

Première chose à lire après avoir construit un graphe : **U ou D**, et les **deux nombres**. Ils doivent correspondre à ce que vous attendiez.

Nœuds, liens, attributs

```
vcount(got); ecount(got)
#> [1] 127
#> [1] 550
V(got)[1:5]
#> + 5/127 vertices, named, from 3874741:
#> [1] Ned          Daenerys     Jon          Littlefinger Arya
E(got)[1:3]
#> + 3/550 edges from 3874741 (vertex names):
#> [1] Ned     --Robert Daenerys--Jorah  Jon     --Sam
V(got)$sex[1:5]
#> [1] "M" "F" "M" "M" "F"
E(got)$weight[1:3]
#> [1] 192 154 121
```

- `V(g)$attribut <- valeurs` ajoute un attribut, dans **l'ordre de `V(g)`**
- `vertex_attr(g)`, `edge_attr(g)` : tous les attributs, sous forme de liste
- `as_data_frame(g, what = "vertices")` : retour au tableau (c'est la version d'**igraph**, qui masque celle de tibble)

Piège : dirigé par défaut

```
g_d <- graph_from_data_frame(got_aretes)   # directed = TRUE par défaut
g_d
#> IGRAPH 81c653a DNW- 127 550 --
#> + attr: name (v/c), weight (e/n)
#> + edges from 81c653a (vertex names):
#>  [1] Ned         ->Robert        Daenerys    ->Jorah
#>  [3] Jon         ->Sam           Ned         ->Littlefinger
#>  [5] Ned         ->Varys         Daenerys    ->Drogo
#>  [7] Ned         ->Arya          Catelyn     ->Robb
#>  [9] Bronn       ->Tyrion        Ned         ->Cersei
#> [11] Cersei      ->Robert        Littlefinger->Varys
#> [13] Shae        ->Tyrion        Ned         ->Catelyn
#> [15] Joffrey     ->Sansa         Ned         ->Pycelle
#> + ... omitted several edges
```

- Aucune erreur, aucun message : le graphe est **dirigé**, `from` → `to`
- Or l'ordre des colonnes ne veut rien dire ici : « Ned–Robert » n'a pas de sens
- Conséquences : `degree(mode = "in")` et `"out"` différents, chemins à sens unique, communautés différentes

```
distances(g_d, "Robert", "Ned", mode = "out")
#>        Ned
#> Robert Inf
```

Robert ne « peut pas atteindre » Ned. Vérifier le **`U`**/**`D`** de l'en-tête.

Piège : les nœuds isolés disparaissent

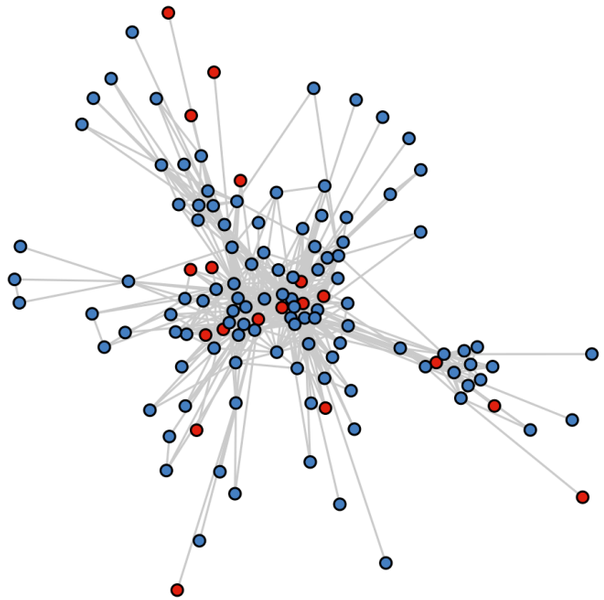
Une liste d'arêtes ne connaît que les nœuds qui ont **au moins un lien**.

```
aretes <- tibble(from = c("A", "B"), to = c("B", "C"))
noeuds <- tibble(name = c("A", "B", "C", "D"), type = c("x", "x", "y", "y"))
vcount(graph_from_data_frame(aretes, directed = FALSE))
#> [1] 3
vcount(graph_from_data_frame(aretes, directed = FALSE, vertices = noeuds))
#> [1] 4
```

- Sans `vertices`, **D** n'existe pas : un gène sans aucune corrélation au-dessus du seuil disparaît du réseau
- Avec `vertices`, tout nom des arêtes **absent** de `noeuds` donne une **erreur** (`Some vertex names in edge list are not listed in vertex data frame`) : c'est une bonne chose
- Compter les nœuds **avant** et **après**

Dessiner : `plot()`

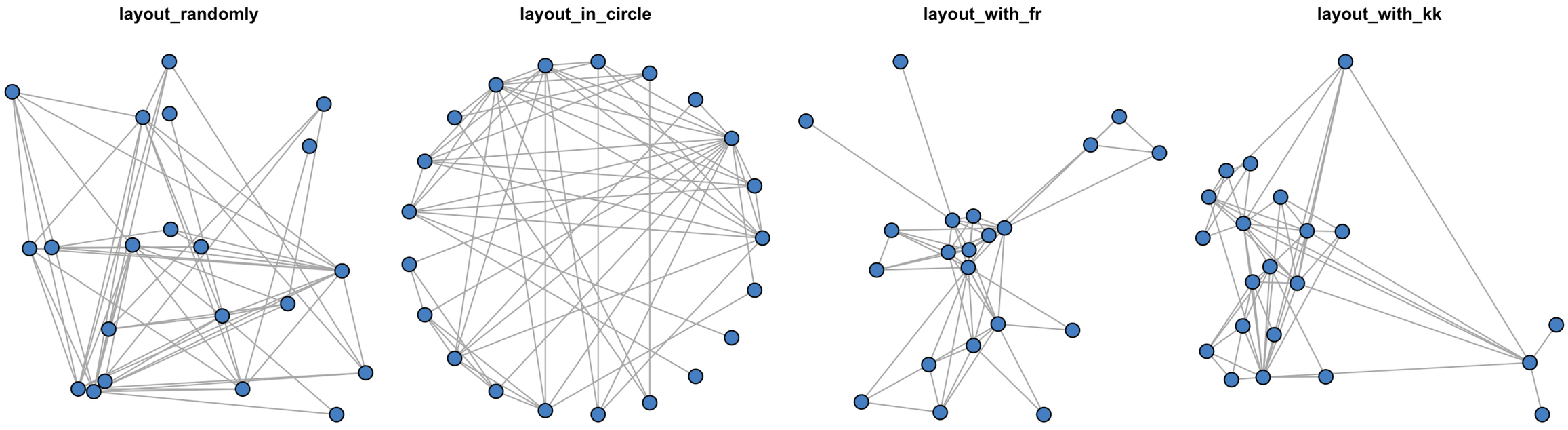
```
set.seed(1)
plot(got, vertex.label = NA, vertex.size = 4,
     vertex.color = if_else(V(got)$sex == "F", "#E30513", "#4A7FC1"), edge.color = "grey80")
```



Options `vertex.*` (couleur, taille, étiquette) et `edge.*` (couleur, largeur, flèches) : `?igraph.plotting`.

Les dispositions (*layouts*)

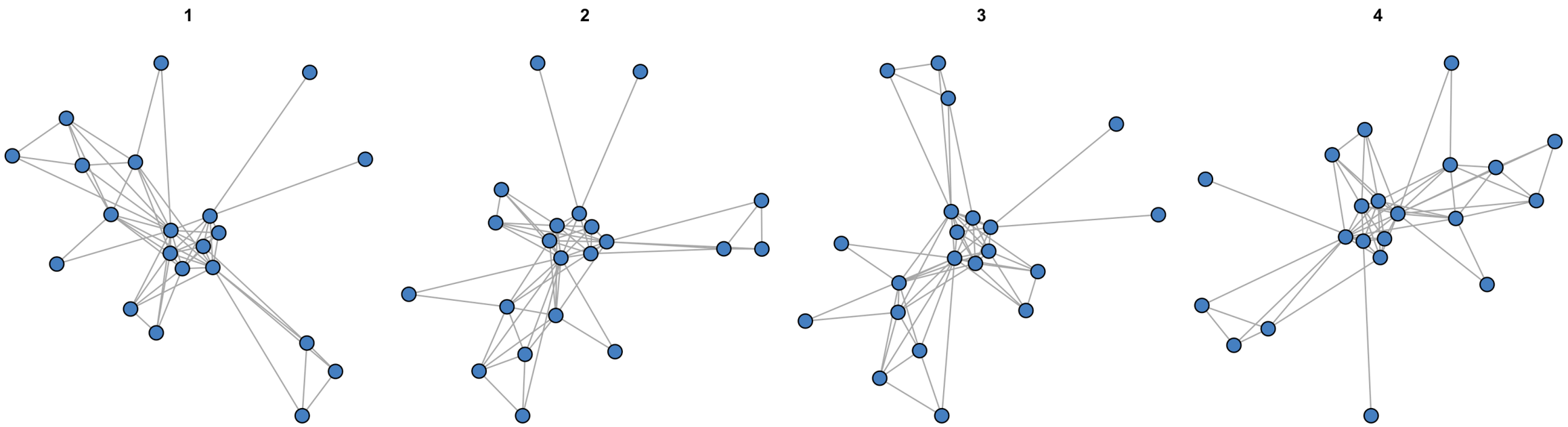
```
par(mfrow = c(1, 4), mar = c(0, 0, 1.5, 0))
for (f in c("layout_randomly", "layout_in_circle", "layout_with_fr", "layout_with_kk")) {
  plot(sw, layout = get(f), vertex.label = NA, vertex.size = 8, vertex.color = "#4A7FC1", main = f
}
```



- Une disposition = un algorithme qui donne des **coordonnées** à chaque nœud
- `layout_with_fr` (Fruchterman-Reingold), `layout_with_kk` : les liens tirent, les nœuds se repoussent
- **Le même graphe**, quatre images : la position n'est **pas** une donnée

Piège : une image différente à chaque fois

```
par(mfrow = c(1, 4), mar = c(0, 0, 1.5, 0))
for (i in 1:4) plot(sw, layout = layout_with_fr, vertex.label = NA, vertex.size = 8, vertex.color
```



- Les algorithmes de disposition partent d'une position **aléatoire** : `set.seed()`, ou calculer `l <- layout_with_fr(g)` **une fois** et le réutiliser
- Deux nœuds **proches à l'écran** ne sont pas forcément liés ; un nœud **au centre** n'est pas forcément central
- Une figure de réseau **illustre** ; les conclusions viennent des **calculs** (section 5)

✋ À vous : lire un objet igraph

```
aretes <- tibble(from = c("TF1", "TF1", "TF2", "TF2", "geneA"),
                 to   = c("geneA", "geneB", "geneB", "TF2", "TF1"),
                 effet = c(1, -1, 1, 1, -1))
g <- graph_from_data_frame(aretes)
g
```

1. Que contient la première ligne de `g` (lettres et nombres) ?
2. `degree(g, "TF2")` ? Et `degree(g, "TF2", mode = "out")` ?
3. Quelle ligne de `aretes` est une boucle ? Que dit-elle en biologie ?

1. `DN--` : **dirigé** (défaut), nommé ; pas de `W` car l'attribut s'appelle `effet`, pas `weight`. **4 nœuds, 5 liens**
2. Sortants : TF2→geneB et TF2→TF2, soit **2** ; entrant : la boucle seule, **1**. `degree()` additionne les deux : **3** (la boucle compte deux fois)
3. `TF2 → TF2` : une **autorégulation** (le facteur active son propre gène)

TP1

4

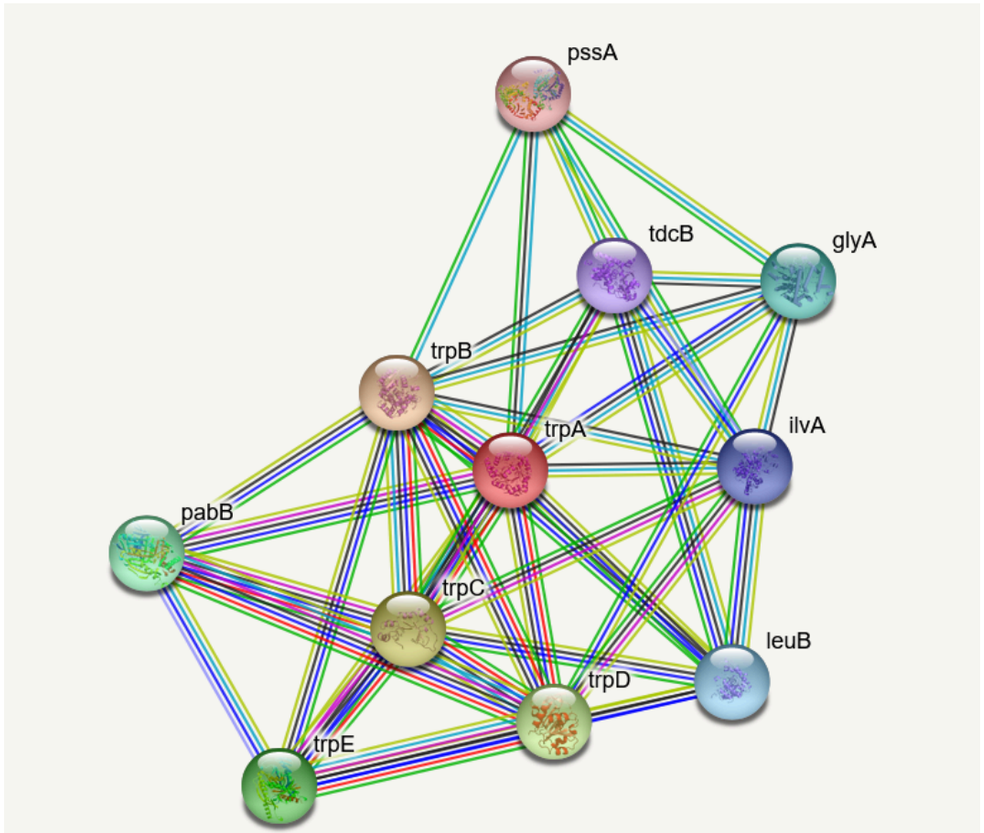
LES RÉSEAUX EN BIOLOGIE

Un panorama

Réseau	Nœuds	Liens	Dirigé ?	D'où viennent les liens ?
Interactions protéiques (PPI)	protéines	liaison physique	non	double-hybride, co-IP, prédictions
Régulation	facteurs, gènes	régulation	**oui**	ChIP-seq, perturbations, inférence
Métabolique	métabolites, réactions	conversion	**oui**	biochimie, génomes annotés
Co-expression	gènes	corrélation	non	données d'expression, **calculé**
Maladies	gènes, maladies	association	non, bipartite	génétique, littérature

Certains liens sont **mesurés**, d'autres **prédits**, d'autres **calculés** à partir de vos données. Même question qu'au module bases de données : d'où vient le lien, et que vaut-il ?

Les interactions protéine-protéine



STRING : une couleur de lien par type de preuve.

Expérimentales

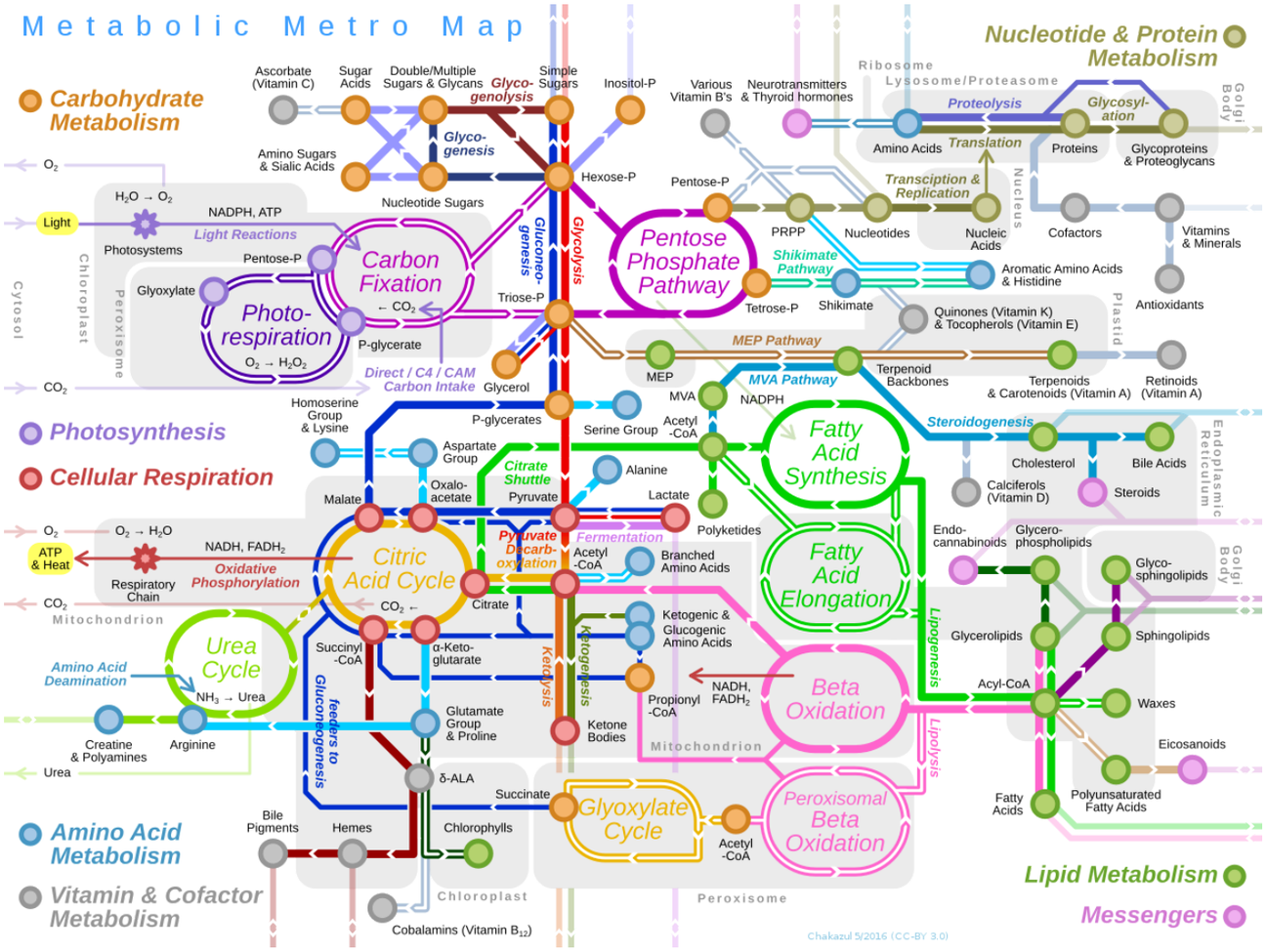
- **double-hybride** chez la levure (*yeast two-hybrid*) : deux protéines qui se lient reconstituent un facteur de transcription
- **co-immunoprécipitation** + spectrométrie de masse : des **complexes**
- ChIP : protéine-ADN

Prédites

- contexte génomique (fusion de domaines, voisinage conservé, profils phylogénétiques), co-expression, **fouille de texte**

STRING combine le tout en un **score de confiance** (0 à 1) : https://string-db.org

Les réseaux métaboliques



Le métabolisme en plan de métro (Wikimedia Commons, *Metabolic Metro Map*). Bases : KEGG, Reactome, MetaCyc.

Dirigés, souvent **bipartites** (métabolites ↔ réactions). Les métabolites très partagés (ATP, H_2O, NADH) relient tout à tout : on les retire souvent avant d'analyser.

Les réseaux de co-expression

Deux gènes sont liés si leurs expressions **varient ensemble** d'un échantillon à l'autre.

1. Une matrice d'expression : gènes × échantillons
2. Une **corrélation** pour chaque paire de gènes
3. Un **seuil** (ou une transformation en puissance, comme WGCNA) : corrélation → lien
4. Un graphe : on y cherche des **modules** et des **hubs**

> Contrairement à une PPI, ce réseau est **calculé à partir de vos données** : chaque choix (gènes retenus, corrélation, seuil) change le graphe. Et une corrélation n'est ni une interaction ni une régulation.

Les données : `airway`, encore

```
coexpr <- read_tsv("/home/public/reseaux/airway_coexpr.tsv")
```

```
coexpr |> dplyr::select(1:6)
#> # A tibble: 150 × 6
#>   gene            symbole chromosome statut N61311_untrt N61311_trt
#>   <chr>           <chr>   <chr>      <chr>         <dbl>      <dbl>
#> 1 ENSG00000129824 RPS4Y1  Y          non DE        12.3       12.2
#> 2 ENSG00000229807 XIST    X          non DE         6.73       6.94
#> 3 ENSG00000123243 ITIH5   10         non DE        14.5       13.5
#> 4 ENSG00000211445 GPX3    5          induit        10.1       14.1
#> 5 ENSG00000101347 SAMHD1  20         induit        10.8       14.2
#> 6 ENSG00000096060 FKBP5   6          induit         8.78      12.4
#> # ℹ 144 more rows
```

- Les **150 gènes les plus variables** d'`airway` (expression stabilisée `vst` de DESeq2), 8 échantillons nommés `donneur_traitement`
- `statut` : induit, réprimé ou non DE par la dexaméthasone (`padj` < 0.05, |log2FC| > 1)
- `chromosome` : on en aura besoin…

De l'expression au graphe

Quatre gènes, huit échantillons : quatre donneurs (D1 à D4), non traités (−) et traités (+) ; expression `vst`.

expression

gène	D1−	D1+	D2−	D2+	D3−	D3+	D4−	D4+
FKBP5	8.8	12.4	9	12.5	9.5	12	8.4	12.2
TSC22D3	9.1	12.3	9.3	12.2	8.9	10.6	9.4	12.4
VCAM1	10.1	7.9	10.2	8	10.4	8.2	9.8	7.5
RPS4Y1	12.3	12.2	12	12	12.2	12	6.7	6.9

De l'expression au graphe

Une corrélation de Pearson par paire : `cor(t(m))`. *FKBP5* et *TSC22D3* montent ensemble (r = 0.92) ; *VCAM1* varie **en sens inverse** (r = -0.95 avec *FKBP5*).

corrélations

gène	FKBP5	TSC22D3	VCAM1	RPS4Y1
FKBP5	1	0.92	-0.95	0.1
TSC22D3	0.92	1	-0.96	-0.14
VCAM1	-0.95	-0.96	1	0.18
RPS4Y1	0.1	-0.14	0.18	1

De l'expression au graphe

Un **seuil** : r > 0.8 → 1, sinon 0. La diagonale est mise à 0 (pas de boucle).

corrélations

gène	FKBP5	TSC22D3	VCAM1	RPS4Y1
FKBP5	1	0.92	-0.95	0.1
TSC22D3	0.92	1	-0.96	-0.14
VCAM1	-0.95	-0.96	1	0.18
RPS4Y1	0.1	-0.14	0.18	1



adjacence

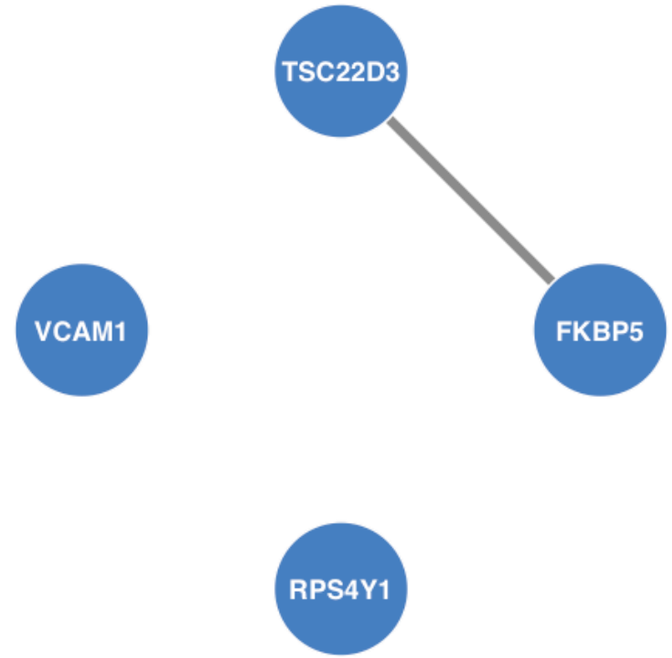
gène	FKBP5	TSC22D3	VCAM1	RPS4Y1
FKBP5	0	1	0	0
TSC22D3	1	0	0	0
VCAM1	0	0	0	0
RPS4Y1	0	0	0	0

De l'expression au graphe

`graph_from_adjacency_matrix(adj, mode = "undirected")` : **un seul lien**. *VCAM1* (anticorrélé) et *RPS4Y1* (chromosome Y) restent **isolés**.

adjacence

gène	FKBP5	TSC22D3	VCAM1	RPS4Y1
FKBP5	0	1	0	0
TSC22D3	1	0	0	0
VCAM1	0	0	0	0
RPS4Y1	0	0	0	0

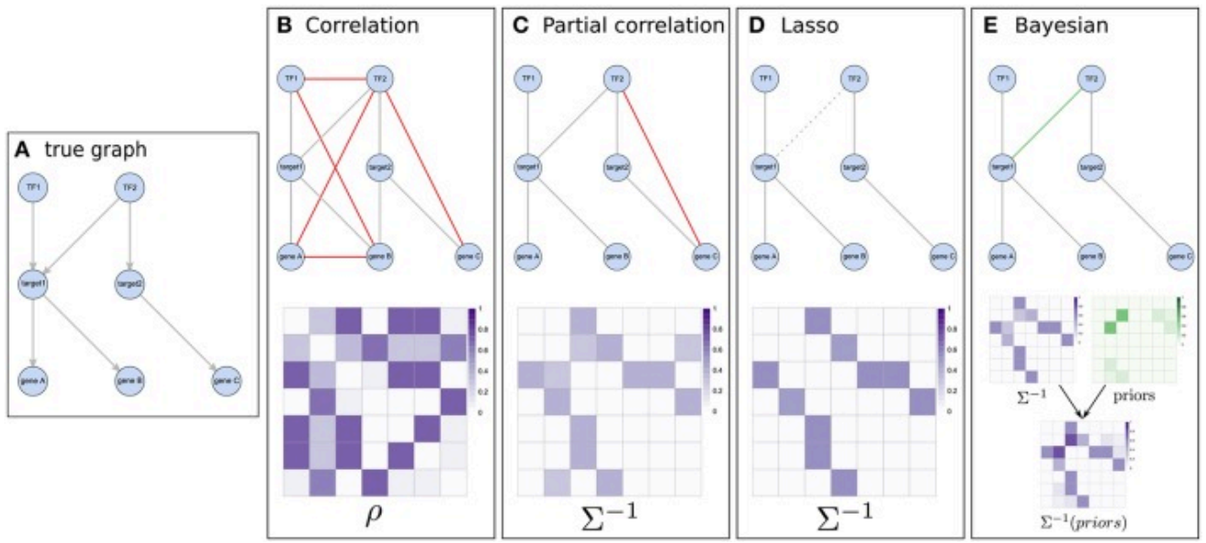


Le réseau complet

```
m <- coexpr |> dplyr::select(symbole, contains("_")) |> column_to_rownames("symbole") |> as.matrix
r <- cor(t(m))                          # 150 × 150 corrélations
adj <- (r > 0.8) * 1; diag(adj) <- 0
g_co <- graph_from_adjacency_matrix(adj, mode = "undirected")
V(g_co)$statut <- coexpr$statut      # même ordre que les lignes de m
g_co
#> IGRAPH 10502a8 UN-- 150 1622 --
#> + attr: name (v/c), statut (v/c)
#> + edges from 10502a8 (vertex names):
#>  [1] RPS4Y1--DDX3Y           RPS4Y1--USP9Y           RPS4Y1--KDM5D
#>  [4] RPS4Y1--ENSG00000131002 RPS4Y1--ENSG00000184674 RPS4Y1--NLGN4Y
#>  [7] RPS4Y1--ZFY             RPS4Y1--UTY             XIST  --GAS7
#> [10] ITIH5 --A2M             ITIH5 --RARRES1         ITIH5 --EGFL6
#> [13] ITIH5 --RCAN2           ITIH5 --OGN             ITIH5 --ALDH1A1
#> [16] ITIH5 --GFRA1           ITIH5 --AHNAK2          ITIH5 --PCDHGA12
#> [19] GPX3  --SAMHD1          GPX3  --FKBP5           GPX3  --SPARCL1
#> [22] GPX3  --GLUL            GPX3  --TSC22D3         GPX3  --MAOA
#> + ... omitted several edges
```

150 gènes, 1622 liens, densité 0.145, 3 gènes isolés. Le `contains("_")` ne garde que les 8 colonnes d'échantillons.

Le problème de l'inférence



Un même vrai réseau (A), quatre méthodes d'inférence, quatre graphes différents.

- Corrélation **marginale** : A et C corrélés parce que tous deux régulés par B ; un lien **indirect**
- Corrélation **partielle**, lasso, réseaux bayésiens : tentent de retirer les liens indirects
- Aucune méthode ne retrouve bien le vrai réseau à partir de l'expression seule (défi DREAM5, Marbach *et al.*, 2012)

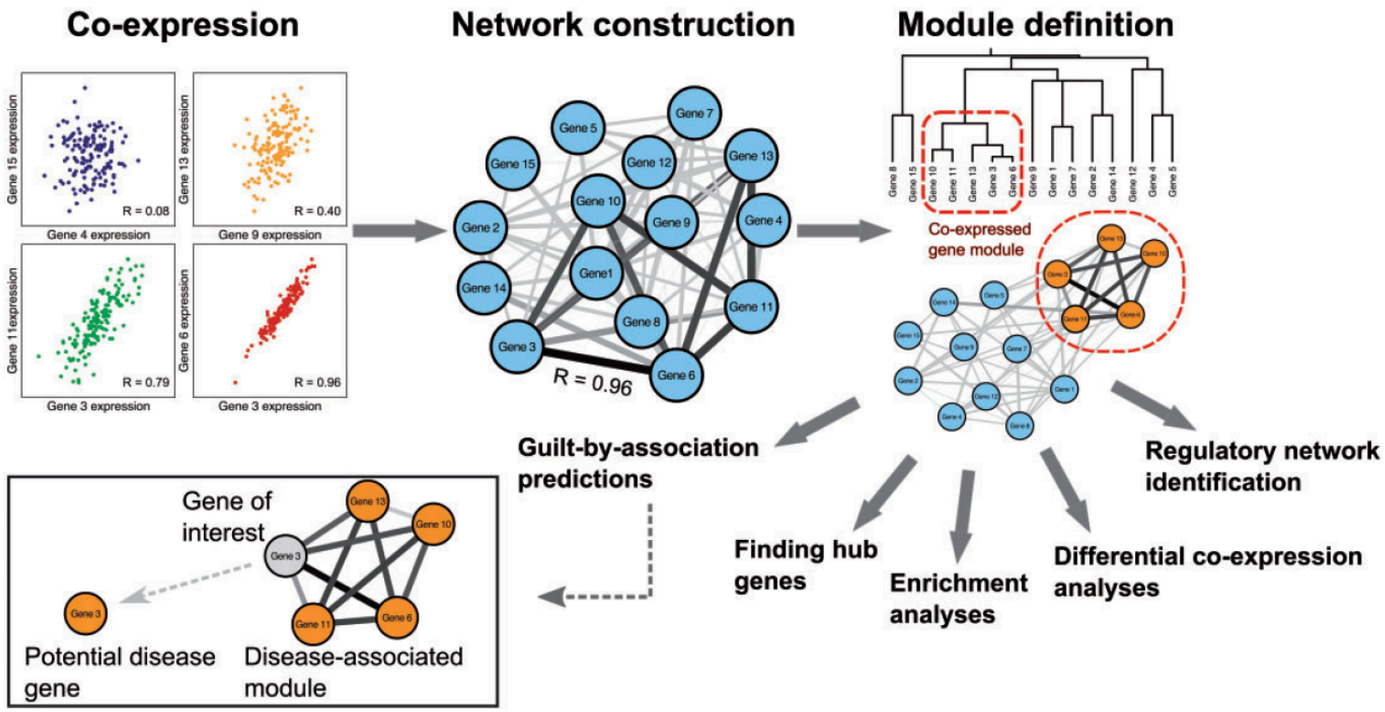
Piège : 8 échantillons

Avec 8 échantillons, de **fortes corrélations apparaissent par hasard**.

```
set.seed(1)
bruit <- matrix(rnorm(150 * 8), nrow = 150)   # 150 « gènes » sans aucun lien réel
r_bruit <- cor(t(bruit))
sum(r_bruit[upper.tri(r_bruit)] > 0.8)
#> [1] 93
```

- 93 liens « r > 0.8 » entre gènes **indépendants**, sur 11 175 paires
- Avec n = 8, une corrélation fluctue énormément : il faut de **nombreux** échantillons (WGCNA recommande au moins une quinzaine à une vingtaine)
- Ici, ce qui sauve `airway` : un **traitement** et des **donneurs** qui structurent fortement l'expression

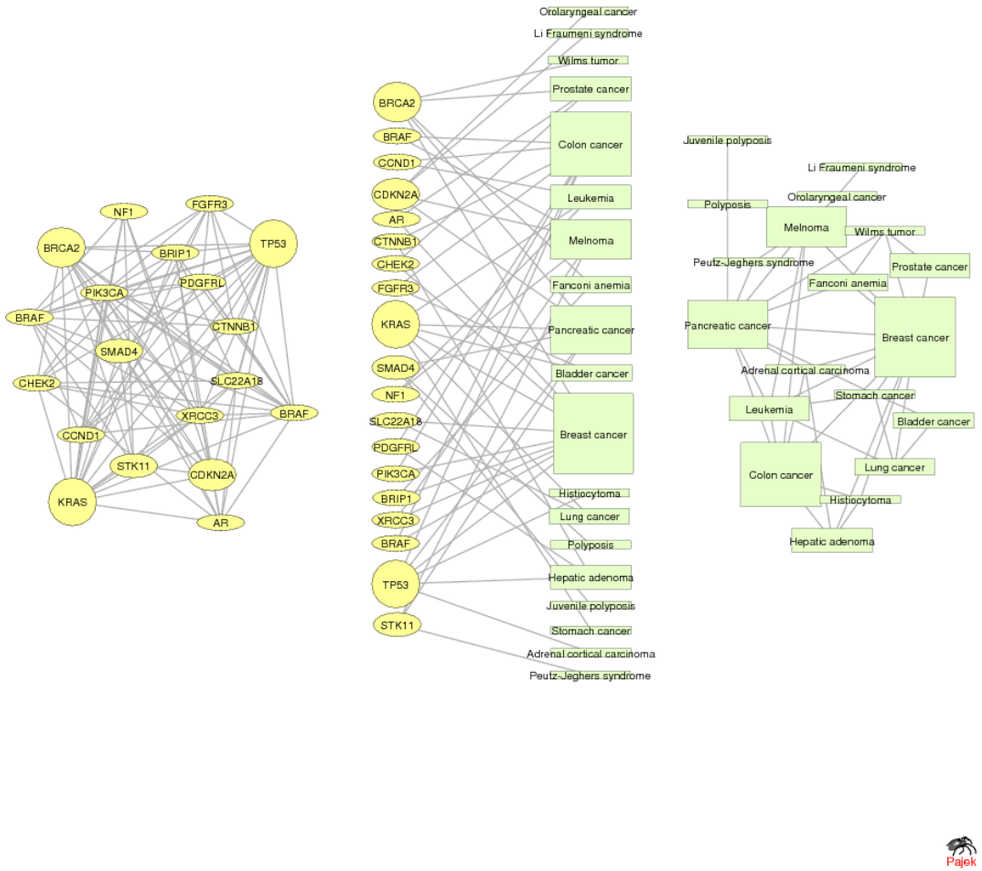
Ce qu'on fait d'un réseau de co-expression



- **Modules** de gènes co-exprimés, reliés à un phénotype
- **Culpabilité par association** (*guilt by association*) : un gène inconnu dans un module de gènes de la réponse immunitaire y participe probablement
- **Hubs** : candidats régulateurs ou biomarqueurs

van Dam *et al.* (2018), *Briefings in Bioinformatics.*

Le réseau des maladies humaines



Goh *et al.* (2007), *PNAS*.

Goh *et al.* (2007) : un graphe **bipartite** gènes ↔ maladies (OMIM), et ses deux **projections** :

- **réseau des maladies** : deux maladies liées si elles partagent un gène
- **réseau des gènes** : deux gènes liés s'ils sont associés à la même maladie

Les gènes d'une même maladie interagissent plus souvent entre eux qu'au hasard : des **modules de maladie**.

5

PROPRIÉTÉS TOPOLOGIQUES

Le degré

Le **degré** k_i d'un nœud : son nombre de liens.

```
sort(degree(got), decreasing = TRUE)[1:6]
#>     Ned  Tyrion Catelyn  Robert    Robb  Cersei
#>      57      41      37      36      30      29
```

- Dirigé : degré **entrant** (`mode = "in"`) + degré **sortant** (`"out"`) = degré total
- Pondéré : la **force** (*strength*) = somme des poids

```
sort(strength(got), decreasing = TRUE)[1:6]
#>      Ned   Tyrion  Catelyn   Robert Daenerys      Jon
#>     1290      708      584      563      535      535
```

Le degré n'est qu'un `count()`

Le degré n'est qu'un `count()`

La liste d'arêtes du graphe jouet.

arêtes

from	to
A	B
A	C
B	C
C	D

Le degré n'est qu'un `count()`

`pivot_longer(c(from, to))` : une ligne par **extrémité** de lien.

arêtes

from	to
A	B
A	C
B	C
C	D



format long

lien	extrémité	nœud
1	from	A
1	to	B
2	from	A
2	to	C
3	from	B
3	to	C
4	from	C
4	to	D

Le degré n'est qu'un `count()`

`count(nœud)` : le nombre d'apparitions de chaque nœud. C'est **exactement** `degree()`.

format long

lien	extrémité	nœud
1	from	A
1	to	B
2	from	A
2	to	C
3	from	B
3	to	C
4	from	C
4	to	D



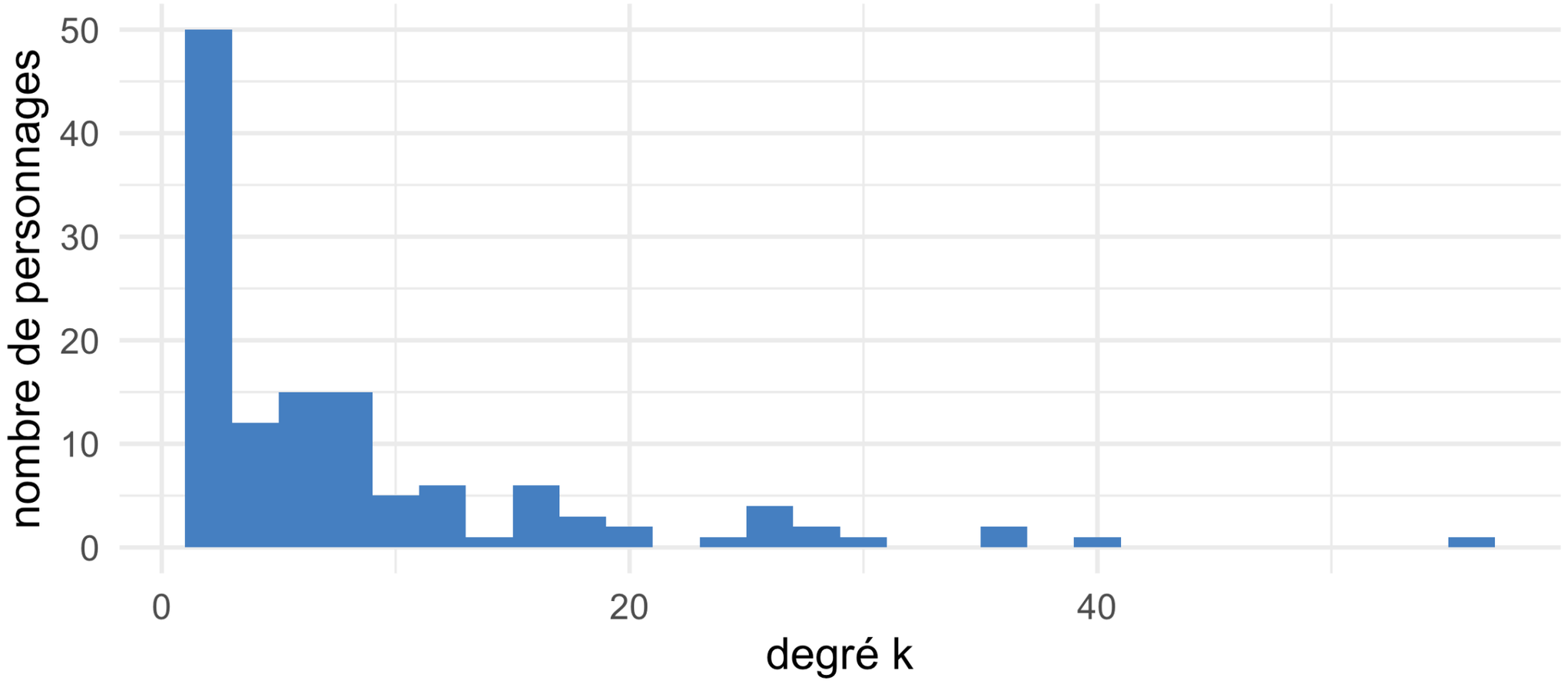
degré

nœud	degré
A	2
B	2
C	3
D	1

```
degree(g_jouet)
 A B C D
2 2 3 1
```

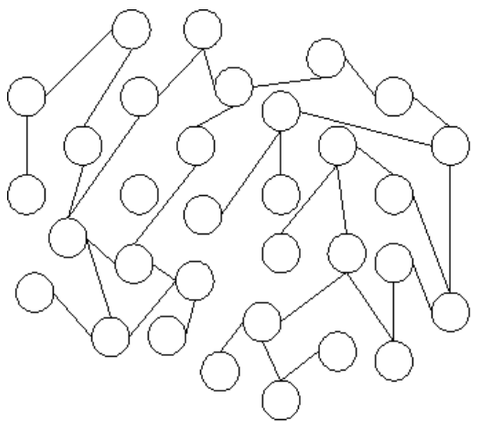
La distribution des degrés

```
tibble(k = degree(got)) |>
  ggplot(aes(k)) + geom_histogram(binwidth = 2, fill = "#4A7FC1") +
  labs(x = "degré k", y = "nombre de personnages")
```

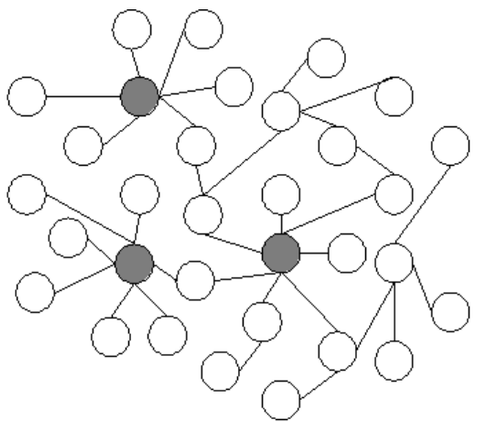


P(k) = N_k / N : la probabilité qu'un nœud pris au hasard ait le degré k. Beaucoup de personnages avec **peu** de liens, quelques-uns avec **beaucoup** : 50 personnages ont au plus 3 liens, Ned en a 57.

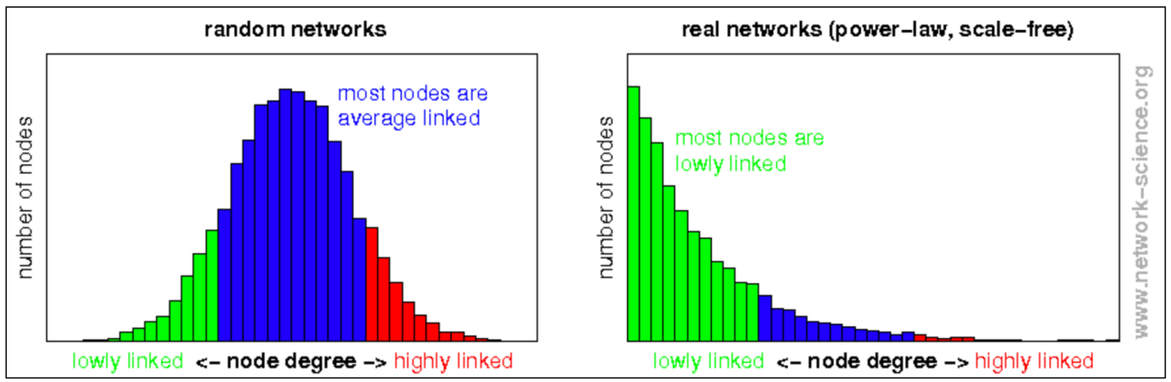
Réseaux aléatoires et invariants d'échelle



(a) Random network



(b) Scale-free network



- **Aléatoire** (Erdős-Rényi) : chaque paire liée avec la même probabilité ; degrés **groupés** autour de la moyenne, pas de hub
- **Invariant d'échelle** (*scale-free*) : $P(k) \sim k^{-\gamma}$, une **loi de puissance** ; beaucoup de nœuds peu liés, quelques **hubs** très liés
- Barabási et Albert (1999) : la **croissance** plus l'**attachement préférentiel** (les riches s'enrichissent) produisent une loi de puissance

Les hubs

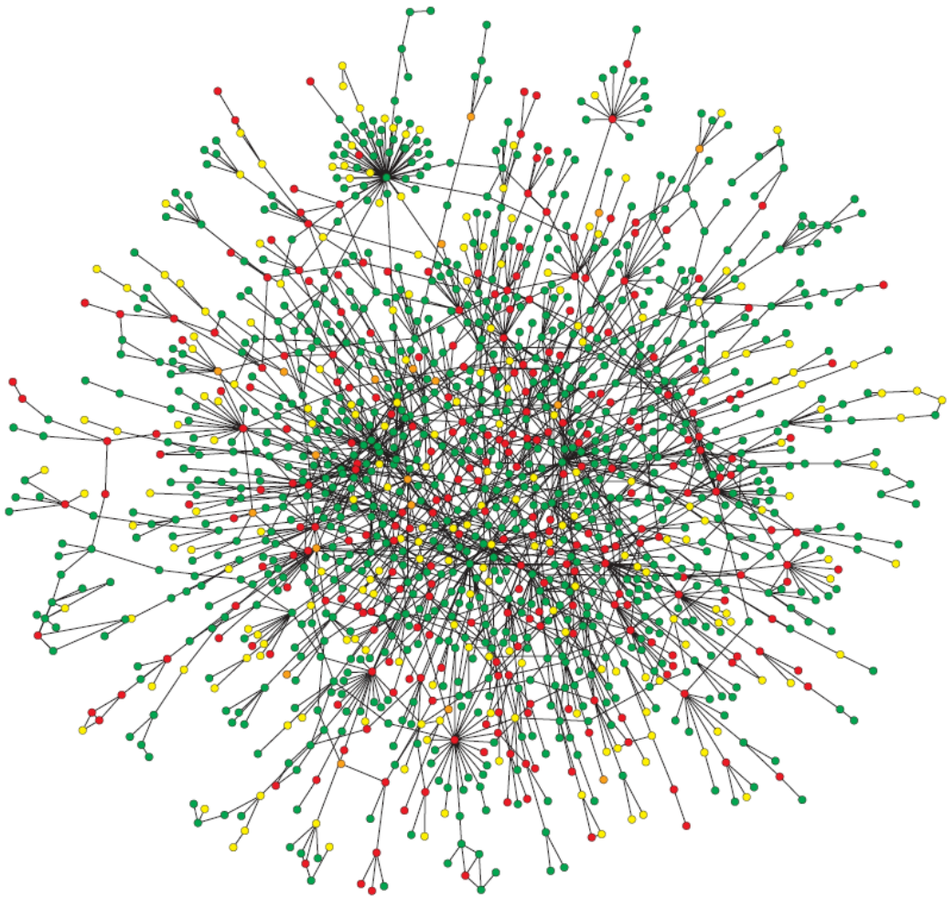


Figure 2 | **Yeast protein interaction network.** A map of protein–protein interactions[18] in *Saccharomyces cerevisiae*, which is based on early yeast two-hybrid measurements[23], illustrates that a few highly connected nodes (which are also known as hubs) hold the network together. The largest cluster, which contains ~78% of all proteins, is shown. The colour of a node indicates the phenotypic effect of removing the corresponding protein (red = lethal, green = non-lethal, orange = slow growth, yellow = unknown). Reproduced with permission from REF. 18 © Macmillan Magazines Ltd.

Réseau d'interactions protéiques de *S. cerevisiae* (Jeong *et al.*, 2001). Couleur = effet de la délétion du gène :

- **rouge** : létal ; **vert** : non létal
- **orange** : croissance lente ; **jaune** : inconnu

Les protéines les plus connectées sont plus souvent **essentielles** : la règle « centralité-létalité ».

Un réseau invariant d'échelle **résiste** aux pannes au hasard, mais est **fragile** face à une attaque ciblée des hubs.

Nuancer : des hubs… ou des gènes très étudiés ?

- Une protéine **célèbre** (TP53, BRCA1) est testée dans plus d'expériences : elle a plus d'interactions **connues**, pas forcément plus d'interactions **réelles**
- Le double-hybride et la co-IP ont leurs biais (faux positifs, protéines membranaires mal détectées)
- Broido et Clauset (2019), sur près de 1 000 réseaux réels : les lois de puissance **strictes** sont **rares** ; « queue lourde » est plus juste

> Le même biais qu'au module bases de données : les gènes déjà étudiés sont les mieux annotés. Un hub d'un réseau **issu de la littérature** est d'abord un gène **beaucoup publié**.

Chemins et distances

- **Chemin** : une suite de nœuds où chacun est lié au suivant
- **Distance** (plus court chemin, géodésique) : le nombre de liens du chemin le plus court ; **infinie** si les nœuds sont dans deux composantes séparées
- Dirigé : on suit le **sens** des flèches, la distance de A à B peut différer de celle de B à A
- **Diamètre** : la plus grande distance ; **longueur moyenne** des plus courts chemins

```
shortest_paths(sw, from = "LUKE", to = "DARTH VADER", weights = NA)$vpath
#> [[1]]
#> + 3/21 vertices, named, from 0e628ee:
#> [1] LUKE        LEIA        DARTH VADER
diameter(sw, weights = NA); mean_distance(sw, weights = NA)
#> [1] 3
#> [1] 1.914286
```

Piège : les poids sont des distances

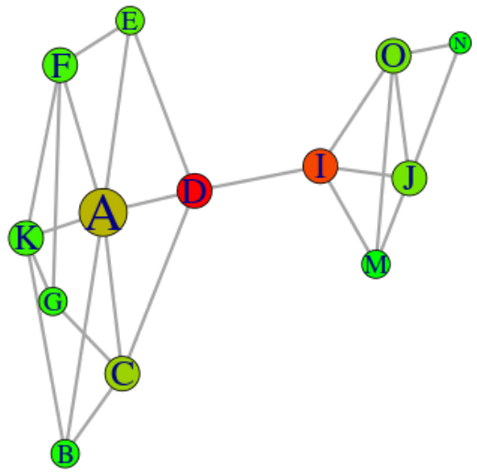
```
diameter(got, weights = NA)     # en nombre de liens
#> [1] 6
diameter(got)                   # avec E(got)$weight, utilisé automatiquement
#> [1] 24
```

- Dès qu'un attribut s'appelle `weight`, `distances()`, `shortest_paths()`, `diameter()`, `betweenness()`… l'utilisent comme une **longueur**
- Ici le poids est un nombre d'interactions : **plus** il est grand, plus les personnages sont **proches**. igraph le lit à l'envers

```
shortest_paths(got, "Jon", "Daenerys", weights = NA)$vpath[[1]]
#> + 3/127 vertices, named, from 3874741:
#> [1] Jon      Ned      Daenerys
shortest_paths(got, "Jon", "Daenerys")$vpath[[1]]
#> + 4/127 vertices, named, from 3874741:
#> [1] Jon           Brandon Stark Catelyn       Daenerys
```

Le « plus court » chemin pondéré **évite** Ned, le personnage le plus lié aux deux. `weights = NA` (ignorer), ou une vraie distance : `1 / weight`.

La centralité d'intermédiarité



Les nœuds rouges relient deux groupes : faible degré, forte intermédiarité.

Betweenness : le nombre de plus courts chemins qui **passent par** un nœud. Un nœud **pont** entre deux modules, même avec peu de liens.

```
sort(betweenness(got, weights = NA), decreasing = TRU
#>       Ned    Tyrion   Catelyn    Robert  Daenerys
#> 2364.5574 1271.4956 1041.7014  860.8102  798.4258
sort(betweenness(got), decreasing = TRUE)[1:5]
#>       Ned     Jorah     Aerys   Catelyn    Tyrion
#> 1599.5311 1220.4734  796.5135  778.6164  767.3571
```

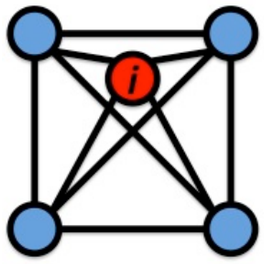
Avec les poids lus comme des distances, Jorah, Aerys et Catelyn passent devant Tyrion : un classement **faux** sans aucun message.

Le coefficient de *clustering*

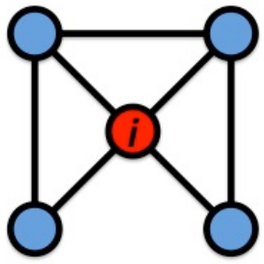
Pour un nœud i de degré k_i : quelle proportion de ses voisins sont **liés entre eux** ?

$$C_i = \frac{2e_i}{k_i(k_i - 1)}$$

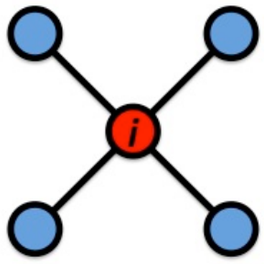
e_i = nombre de liens entre les voisins de i ; $C_i \in [0, 1]$.



$C_i = 1$



$C_i = 1/2$



$C_i = 0$

« Les amis de mes amis sont mes amis. »

```
transitivity(got, type = "global")
#> [1] 0.3818514
transitivity(got, type = "local", vids = c("Ned"
#>       Ned       Jon     Drogo
#> 0.1754386 0.3600000 0.3846154
```

Un réseau aléatoire de même densité aurait C ≈ 0.07.

Le petit monde

- **Courtes distances** : distance moyenne 2.65 dans Game of Thrones, 1.91 dans Star Wars
- **Fort *clustering*** : 0.38 contre 0.07 attendu au hasard
- Watts et Strogatz (1998) : quelques liens « raccourcis » dans un réseau très local suffisent à rapprocher tout le monde

> Biologie : un signal ou une perturbation atteint presque toute la cellule en quelques étapes ; les « six degrés de séparation » des réseaux sociaux.

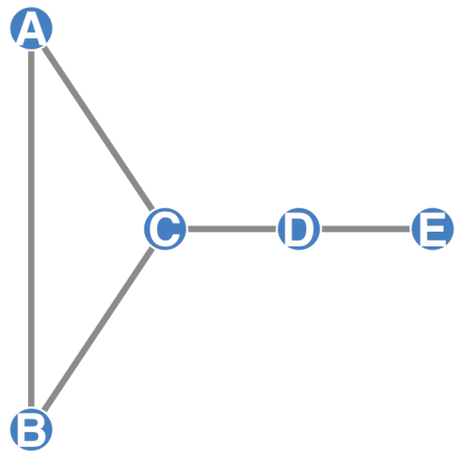
Connexité et composantes

```
components(g_co)$no
#> [1] 5
sort(components(g_co)$csize, decreasing = TRUE)
#> [1] 138   9   1   1   1
```

- Une **composante** : un ensemble de nœuds tous reliés par des chemins
- Entre deux composantes, la distance est **infinie** (`Inf`) : `mean_distance()` ignore ces paires sans le dire

> `components()` existe aussi dans le package `generics`, que charge Bioconductor : chargé après igraph (par exemple `library(DESeq2)`), il la **masque** (`no applicable method for 'components'`). Écrire `igraph::components()`.

✋ À vous : prédire des descripteurs



1. Le degré de chaque nœud ? La densité ?
2. La distance de A à E ? Le diamètre ?
3. Le *clustering* local de C ?
4. Quel nœud a la plus forte intermédiarité ?

1. A 2, B 2, C **3**, D 2, E 1 ; densité = 5 / 10 = **0,5**
2. A–C–D–E : **3** ; diamètre **3** (A ou B à E)
3. Voisins de C : A, B, D ; un seul lien entre eux (A–B) sur 3 possibles : **1/3**
4. **C** (4 chemins passent par lui), puis D (3)

6

MODULES ET COMMUNAUTÉS

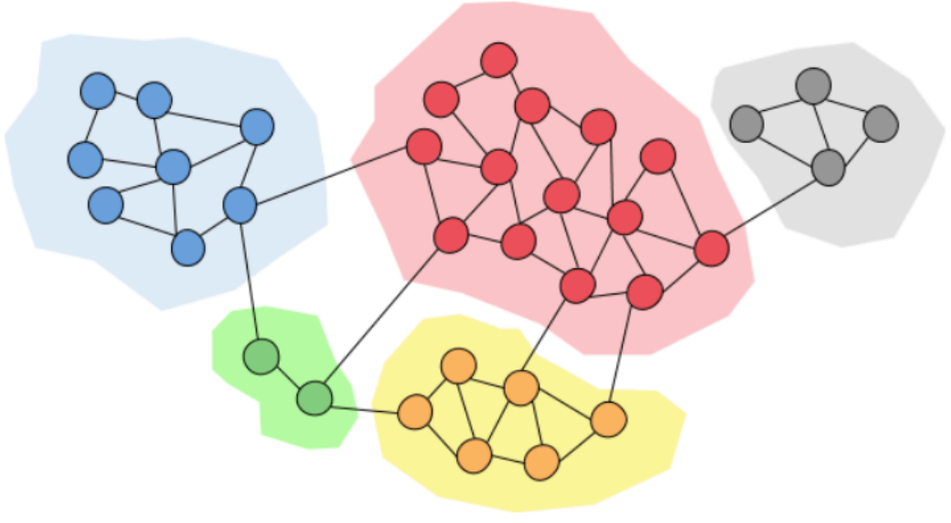
Les cliques

Une **clique** : un sous-graphe **complet**, où chaque nœud est lié à tous les autres.

```
clique_num(got)
#> [1] 10
largest_cliques(got)[[1]]
#> + 10/127 vertices, named, from 3874741:
#>  [1] Jaime        Ned          Robert       Cersei       Catelyn
#>  [6] Littlefinger Joffrey      Hound        Arya         Sansa
length(largest_cliques(got))
#> [1] 7
```

- 7 cliques de taille 10, qui se **chevauchent** : une notion trop stricte (un seul lien manquant suffit à casser la clique)
- Biologie : un complexe protéique où tout le monde se touche

La modularité



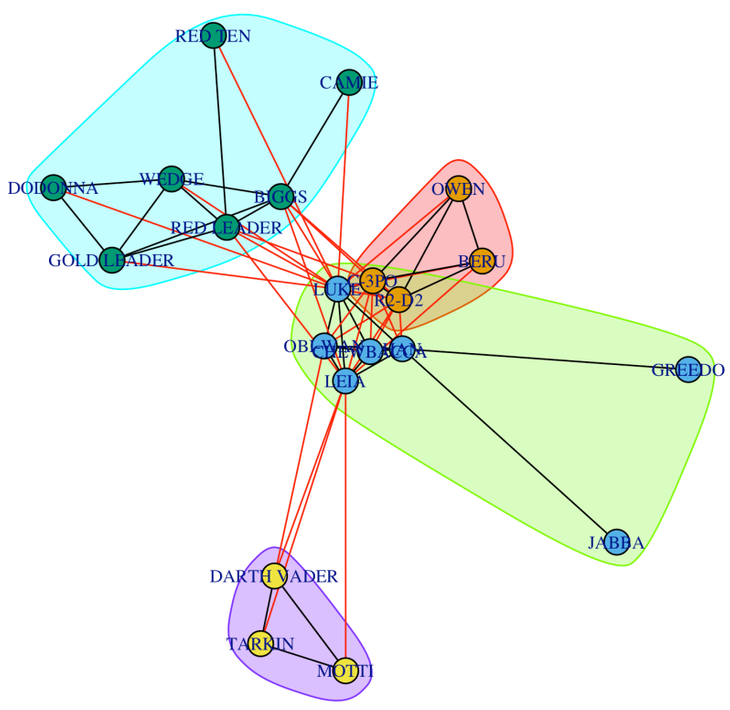
Un **module** (communauté) : beaucoup de liens **à l'intérieur**, peu **vers l'extérieur**.

Modularité Q (Newman et Girvan, 2004) : la fraction des liens internes aux modules, **moins** celle attendue au hasard (degrés conservés). Q > 0,3 : structure nette.

- **Agglomératives** : chaque nœud seul au départ, on fusionne (Louvain, Blondel *et al.*, 2008)
- **Divisives** : tout ensemble, on coupe les liens les plus « intermédiaires » (Girvan-Newman)
- **Marches aléatoires** : un marcheur reste piégé dans son module (walktrap)

Détecter des communautés dans R

```
set.seed(1)
comm <- cluster_louvain(sw)
membership(comm)[c("LUKE", "LEIA", "DARTH VADER", "TARKIN", "BERU", "OWEN")]
#>        LUKE        LEIA DARTH VADER      TARKIN        BERU        OWEN
#>           2           2           4           4           1           1
sizes(comm); modularity(comm)
#> Community sizes
#> 1 2 3 4
#> 4 7 7 3
#> [1] 0.1760294
par(mar = c(0, 0, 0, 0)); plot(comm, sw, vertex.label.cex = 0.7, vertex.size = 8)
```



Les modules de co-expression

```
set.seed(1)
modules <- cluster_louvain(g_co)
V(g_co)$module <- membership(modules)
tibble(module = V(g_co)$module, statut = V(g_co)$statut, chr = coexpr$chromosome) |>
  dplyr::count(module, statut) |>
  pivot_wider(names_from = statut, values_from = n, values_fill = 0) |>
  filter(induit + `non DE` + réprimé > 2)
#> # A tibble: 6 × 4
#>   module     `non DE` réprimé induit
#>   <membrshp>    <int>   <int>  <int>
#> 1 1                 9       0      0
#> 2 2                 1      17      0
#> 3 3                26       4      6
#> 4 4                 0       0     45
#> 5 5                 3       0     26
#> 6 6                 6       3      1
```

Des modules **purs** : induits d'un côté, réprimés de l'autre. Et le module 1, **non DE** : *RPS4Y1*, *DDX3Y*, *USP9Y*, *KDM5D*, *ENSG00000131002*, *ENSG00000184674*, *NLGN4Y*, *ZFY*, *UTY*. Que se passe-t-il ?

Un module… le sexe du donneur

```
coexpr |> filter(symbole %in% c("RPS4Y1", "DDX3Y", "XIST")) |> dplyr::select(symbole, contains("_"
#> # A tibble: 3 × 9
#>   symbole N61311_untrt N61311_trt N052611_untrt N052611_trt N080611_untrt
#>   <chr>          <dbl>      <dbl>          <dbl>       <dbl>          <dbl>
#> 1 RPS4Y1          12.3       12.2           12.0        12.0           12.2
#> 2 XIST             6.73       6.94           6.92        6.73           6.73
#> 3 DDX3Y           10.7       10.6           10.4        10.6           10.8
#> # ℹ 3 more variables: N080611_trt <dbl>, N061011_untrt <dbl>, N061011_trt <dbl>
```

- Les gènes du **chromosome Y** sont éteints chez le donneur N061011, qui exprime *XIST* : c'est une **donneuse**
- Un module parfaitement « réel » et cohérent… qui n'a **rien à voir** avec la dexaméthasone
- *XIST*, **anticorrélé**, n'est pas dans ce module : nous n'avons gardé que les corrélations **positives**

> Un module dit : « ces gènes varient ensemble **dans ces échantillons** ». Le pourquoi (traitement, donneur, sexe, lot de séquençage) est à **chercher**, par exemple par une analyse d'enrichissement.

Piège : le seuil

```
seuils <- c(0.7, 0.8, 0.9)
map_dfr(seuils, function(s) {
  a <- (r > s) * 1; diag(a) <- 0
  g <- graph_from_adjacency_matrix(a, mode = "undirected")
  set.seed(1)
  tibble(seuil = s, liens = ecount(g), isolés = sum(degree(g) == 0),
         composantes = igraph::components(g)$no, modules = length(cluster_louvain(g)))
})
#> # A tibble: 3 × 5
#>   seuil liens isolés composantes modules
#>   <dbl> <dbl>  <int>       <dbl>   <int>
#> 1   0.7  2304      0           1       5
#> 2   0.8  1622      3           5       9
#> 3   0.9   846     23          31      34
```

Le seuil est un **choix**, pas un résultat : il fait varier le nombre de liens, d'isolés, de modules. Montrer que la conclusion **résiste** à plusieurs seuils. WGCNA remplace le seuil par une puissance (r^{β}) choisie pour approcher une loi de puissance (Langfelder et Horvath, 2008).

Piège : la valeur absolue

```
adj_abs <- (abs(r) > 0.8) * 1; diag(adj_abs) <- 0
g_abs <- graph_from_adjacency_matrix(adj_abs, mode = "undirected")
set.seed(1)
table(module = membership(cluster_louvain(g_abs)), coexpr$statut)
#>
#> module induit non DE réprimé
#>      1       0     10        1
#>      2       7     34        5
#>      3      45      0       10
#>      4      26      3        8
#>      5       0      0        1
```

- Avec `abs(r)`, deux gènes **anticorrélés** sont liés : induits et réprimés tombent dans les **mêmes** modules
- Parfois voulu (un régulateur et sa cible réprimée), mais il faut le **savoir** : réseau **signé** ou **non signé**
- Même erreur que mélanger induits et réprimés dans une ORA

Piège : Louvain n'est pas déterministe

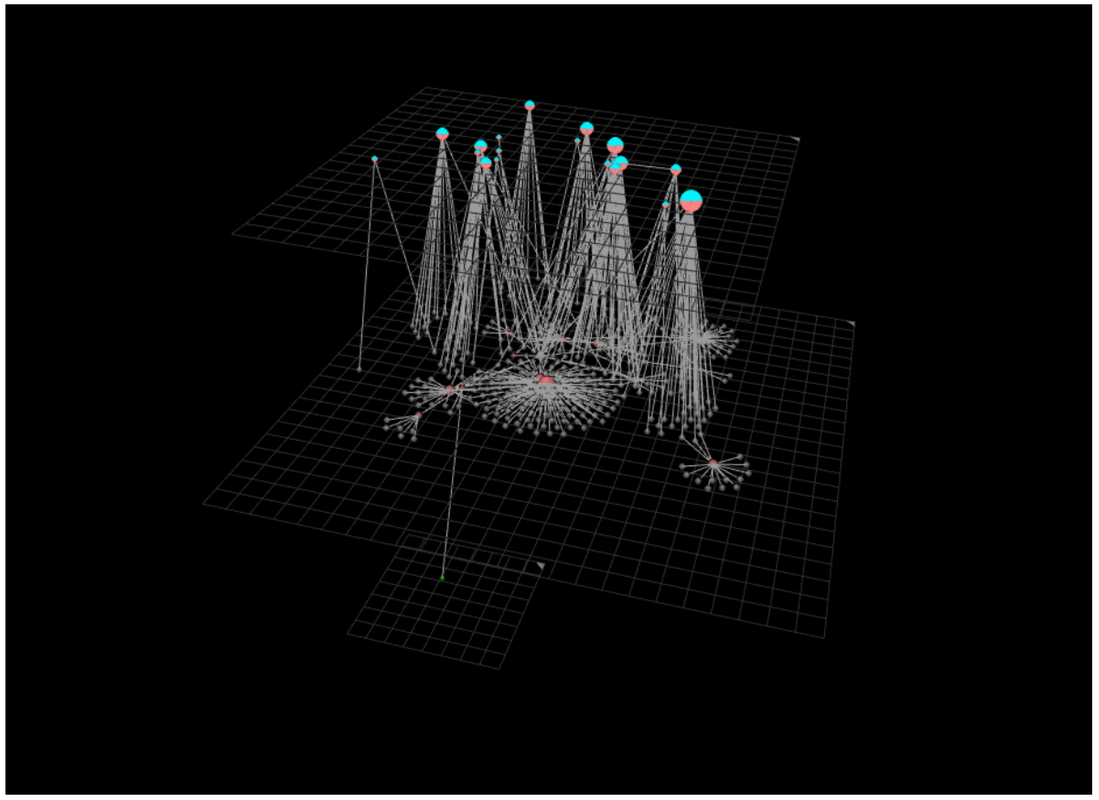
```
tailles <- map(1:5, function(graine) { set.seed(graine); sort(sizes(cluster_louvain(got)), decreas
map_chr(tailles, \(t) paste(t, collapse = " "))
#> [1] "45 24 19 17 16 4 2" "45 24 19 17 16 4 2" "47 23 18 17 16 4 2"
#> [4] "47 28 19 17 16"     "45 24 19 17 16 4 2"
```

- L'algorithme parcourt les nœuds dans un ordre **aléatoire** : la partition peut changer d'une exécution à l'autre
- `set.seed()` pour reproduire ; comparer plusieurs graines pour juger la **stabilité** ; méthodes plus récentes : Leiden (Traag *et al.*, 2019)
- Les numéros de modules sont des **étiquettes** : le module 1 d'une exécution n'est pas le module 1 de la suivante

Au-delà : propagation et multi-omique

Propagation sur le réseau : partir de gènes connus (graines) et diffuser par **marche aléatoire** ; on classe les autres nœuds selon leur proximité. Mieux que les voisins directs (faux négatifs) ou les plus courts chemins (faux positifs).

Réseaux multicouches : PPI + régulation + métabolisme + co-expression, reliés entre eux (OmicsNet, https://www.omicsnet.ca).



Ce qu'on retient des réseaux biologiques

Propriété	Ce que cela veut dire
Creux	peu de liens parmi tous les possibles
Queue lourde	beaucoup de nœuds peu liés, quelques hubs
Petit monde	courtes distances et fort *clustering*
Modulaire	des modules fonctionnels, interprétables par enrichissement
Robuste	résiste aux pannes au hasard, fragile aux attaques des hubs
Dirigé, hiérarchique	cascades de régulation et de signalisation
Multicouche	plusieurs types de liens, plusieurs types de nœuds

✋ À vous : lire une analyse de réseau

Un étudiant construit un réseau de co-expression sur 6 échantillons, avec `abs(cor) > 0.7`, puis écrit :

```
g <- graph_from_data_frame(liens)          # liens : gene1, gene2, weight = |r|
hub <- names(which.max(betweenness(g)))
```

> « Le gène hub est le régulateur central de la réponse au traitement. »

1. Trois problèmes dans le code.
2. Deux objections à la conclusion.

1. Graphe **dirigé** par défaut ; `weight` = |r| lu comme une **distance** (les gènes les plus corrélés sont les plus « loin ») ; `abs()` mélange corrélations positives et **négatives**
2. 6 échantillons : beaucoup de corrélations **dues au hasard**, et le résultat dépend du **seuil** ; une corrélation n'est pas une **régulation** (pas de sens, pas de causalité). Le « hub » peut refléter un donneur, le sexe ou un lot

TP2

Conclusion

- Un **graphe** : des nœuds, des liens, dirigés ou non, pondérés ou non ; choisir la **représentation** est déjà résoudre le problème (Euler, de Bruijn)
- Dans R : `graph_from_data_frame()` (**dirigé par défaut**), `graph_from_adjacency_matrix()` ; lire l'en-tête `U`/`D`, N et L
- Les liens biologiques sont **mesurés**, **prédits** ou **calculés** ; un réseau de co-expression dépend du nombre d'échantillons et du **seuil**
- Degré, distance, intermédiarité, *clustering* : un attribut `weight` est lu comme une **distance**
- Modules : un algorithme parfois **aléatoire**, des modules à **interpréter** (traitement ? donneur ? sexe ?)
- Une figure de réseau **illustre**, elle ne démontre rien

Ressources

- A.-L. Barabási, *Network Science* (2016), libre en ligne : http://networksciencebook.com
- K. Ognyanova, *Network visualization with R* : https://kateto.net/netscix2016.html
- Documentation igraph pour R : https://r.igraph.org
- Koutrouli *et al.* (2020), *A guide to conquer the biological network era using graph theory*, *Frontiers in Bioengineering and Biotechnology*
- Langfelder et Horvath (2008), *WGCNA: an R package for weighted correlation network analysis*, *BMC Bioinformatics*

Données : Game of Thrones (A. Beveridge), Star Wars (E. Gabasova), via `networkdata` (D. Schoch) ; `airway` (Himes *et al.*, 2014 ; GEO GSE52778).